VITAL GUARD:
A REAL TIME REMOTE PATIENT MONITORING SYSYTEM

MINI PROJECT REPORT

Submitted in partial fulfillment of the
Requirements for the award of Bachelor of Technology Degree
In Electronics and Communication Engineering
Of APJ Abdul Kalam Technological University

By

AKHIL M DEEPAK (MBT22EC018/B22EC1209)
GOURI M G (MBT22EC052/B22EC1228)
KULSAM NASREEN P NAZEER (MBT22EC064/B22EC1234)
SIDHARTH H NAIR (MBT22EC095/B22EC1253)

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

MAR BASELIOS COLLEGE OF ENGINEERING & TECHNOLOGY
(Autonomous)
MAR IVANIOS VIDYANAGAR, NALANCHIRA, THIRUVANANTHAPURAM, 695015.

2025

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

MAR BASELIOS COLLEGE OF ENGINEERING & TECHNOLOGY

(Autonomous)

MAR IVANIOS VIDYANAGAR, NALANCHIRA, THIRUVANANTHAPURAM, 695015

CERTIFICATE

This is to certify that this mini project report entitled **"VITAL GUARD: A REAL TIME REMOTE PATIENT MONITORING SYSYTEM"** is a bonafide record of work done by **AKHIL M DEEPAK, GOURI M G, KULSAM NASREEN P NAZEER and SIDHARTH H NAIR** of the sixth semester Electronics and Communication branch towards the partial fulfillment of the requirements for the award of Bachelor of Technology Degree in Electronics and Communication Engineering of APJ Abdul Kalam Technological University.

Guide	Coordinator	Head of the Department
Ms. Teena Rajan **Assistant Professor,** **Dept. of ECE** **MBCET**	**Mr. Anoop K. Johnson,** **Assistant Professor,** **Dept. of ECE** **MBCET**	**Dr. Luxy Mathews** **Associate Professor,** **Dept. of ECE** **MBCET**

ACKNOWLEDGEMENT

With great enthusiasm and pleasure, we are bringing out this mini project report here. We use this opportunity to express our heartiest gratitude for the support and guidance offered to us from various sources during the course of the completion of our project. We are also grateful to **Dr. Luxy Mathews, Associate Professor,** Head of the Department, Electronics and Communication Engineering, for her valuable suggestions. It is our pleasant duty to acknowledge our Mini project Co-coordinator, **Mr. Anoop K. Johnson, Assistant Professor,** in ECE dept., who has guided and given supervision for the project work. Let us express our heartfelt gratitude to our guide, **Ms. Teena Rajan**, **Assistant Professor,** in ECE dept. for helping us all throughout the project. Above all, we owe our gratitude to the **Almighty** for showering abundant blessings upon us. We also express our whole hearted gratefulness to all our classmates who have expressed their views and suggestions about our projects and have helped us during the course of the project. We extend our sincere thanks and gratitude once again to all those who helped us make this undertaking a success.

ABSTRACT

Healthcare has long embraced technology, especially in remote patient monitoring. While systems using Raspberry Pi or Arduino have advanced the field, they often lack comprehensive health tracking, efficient data transmission, and affordability. Vital Guard offers a smarter, cost-effective solution using the ESP32 microcontroller and a glove-based sensor module equipped with MAX30102, AD8232, and DHT11 to measure heart rate, SpO2, ECG, and body temperature. This data is transmitted in real-time to Firebase, with a user-friendly React interface for monitoring. Vital Guard enhances care quality, reduces costs, and offers a compact, low-power design—making it especially valuable for patients with chronic conditions, the elderly, and those in remote areas by enabling early detection and better health outcomes.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION

Vital Guard is an innovative and advanced remote patient monitoring system that brings together the power of cutting-edge sensors, microcontrollers, and cloud technologies to enable continuous, real-time tracking of critical health parameters. The system is designed with a focus on providing reliable, low-power consumption while maintaining ease of use—making it ideal for home-based healthcare applications or remote patient monitoring. The system's versatility ensures that healthcare providers can stay connected with their patients, monitor health metrics from afar, and make timely decisions based on real-time data. This section provides an in-depth look at the core technologies that form the foundation of Vital Guard and how they work together to create a seamless healthcare experience.

At the heart of the Vital Guard system is the ESP32 microcontroller. This small but powerful chip is a dual-core processor equipped with both Wi-Fi and Bluetooth capabilities, allowing it to handle various sensor data inputs and wirelessly transmit it to a cloud platform for remote monitoring. The ESP32 is highly flexible, supporting a wide range of sensors, which makes it the ideal choice for the Vital Guard system. Its low power consumption ensures that the device can run continuously, which is particularly important for wearable or other portable healthcare devices that need to provide real-time data over extended periods. The ESP32 serves as the central hub of the system, processing signals from the various sensors, aggregate the data, and transmit it to a cloud platform in real-time, ensuring that healthcare providers have access to up-to-date information at any moment.

To monitor the heart rate and blood oxygen levels (SpO_2) of the patient, the MAX30102 optical sensor is used. This compact and efficient sensor is equipped with red and infrared LEDs that measure pulse rate and oxygen saturation levels through a process known as photoplethysmography (PPG). The MAX30102 is known for its high sensitivity and ability to function accurately even in a variety of environmental conditions. Its low power consumption makes it an ideal choice for wearable health devices, where constant monitoring is required. The MAX30102 is capable of continuously tracking these vital signs, providing real-time feedback on a patient's cardiovascular health, and sending the data wirelessly to the ESP32 for further processing and transmission. Whether the patient is indoors, outdoors, or in varying

light conditions, the MAX30102 ensures accurate and reliable data for healthcare professionals to analyze.

For ECG (electrocardiogram) monitoring, the system utilizes the AD8232 sensor module. This is an analog front-end module that is specifically designed to measure and amplify the tiny bioelectrical signals that the heart generates during its activity. The signals are often weak and need amplification before they can be processed by other devices. The AD8232 amplifies and filters these signals to ensure that they are clear and precise, providing clean ECG data that helps detect abnormal heart patterns, such as arrhythmias. The ability to detect such irregularities early is crucial in preventing more severe health complications. Once the signals are amplified and filtered by the AD8232, the system transmits the ECG data in real time to the ESP32, where it can be stored and accessed by healthcare providers via the cloud platform.

In addition to heart rate and ECG monitoring, temperature is another vital health parameter that needs to be tracked regularly. To measure body temperature, the Vital Guard system incorporates the DHT11 sensor, a widely-used and cost-effective temperature and humidity sensor. The DHT11 is able to provide accurate and reliable readings of both the ambient temperature and humidity. In the context of patient monitoring, temperature is an important indicator of health, especially when tracking signs of fever or infection. Sudden changes in body temperature can be an early sign of health complications that may require immediate atten- tion. The DHT11 sensor is compatible with the ESP32, which processes the data and trans- mits it wirelessly for analysis, ensuring that healthcare professionals are notified of any tem- perature abnormalities in a timely manner.

Once all the sensor data is collected and processed by the ESP32, it is transmitted to a cloud platform for real-time synchronization and visualization. In the case of Vital Guard, this platform is Firebase, a cloud-based real-time database that facilitates the rapid and seamless transfer of data between the hardware components and the frontend interface. Firebase en- sures that the data is continuously updated and synchronized, so that healthcare providers can monitor patient health in real-time, regardless of their physical location. It also supports the storage of historical data, which allows for long-term health trend tracking and analysis. This feature is particularly valuable for healthcare professionals who need to track the progress of a patient's condition over time, detect any changes in health trends, and make informed deci- sions based on the most recent data.

On the software side, the frontend user interface for VitalGuard is developed using JavaScript and the React framework. React is a widely-used JavaScript library that allows for the creation of dynamic and responsive user interfaces. Through the web-based dashboard, users can easily visualize heart rate graphs, ECG waveforms, temperature trends, and other health metrics in real-time. React's component-based structure allows for modularity, ensuring that each part of the dashboard can be updated or improved independently as needed. This makes the system scalable and maintainable over time, ensuring that VitalGuard can evolve with new features or additional sensors as healthcare needs change. The intuitive design of the frontend interface ensures that healthcare providers can quickly interpret the data and respond to any anomalies or alerts in real-time, without being overwhelmed by complex data.

What makes this real-time display so beneficial is its ability to support healthcare providers in managing multiple patients at once. The system allows caregivers or doctors to remotely monitor several patients from a single dashboard, whether they're at a clinic, in their office, or even from the comfort of their own home. This remote monitoring capability is especially valuable for those who may need to track patients with chronic conditions, elderly patients, or those recovering from surgery, as it provides a way to stay on top of health trends and intervene quickly when necessary. If something unusual happens—like a sudden spike in temperature or a heart rhythm irregularity—the system will instantly alert the healthcare provider, enabling them to take quick action and address the situation before it escalates.

In conclusion, VitalGuard is a comprehensive solution that combines the power of advanced sensors, microcontrollers, and cloud technologies to offer continuous, real-time health monitoring. By providing real-time data visualization, cloud integration, and remote monitoring capabilities, the system ensures that patients receive timely care, even when they're not physically present with their healthcare providers. The use of reliable, low-power sensors ensures that patients can be monitored over extended periods, while the cloud-based platform guarantees that data is available at all times. With the flexibility to expand or upgrade the system as needed, VitalGuard represents a future-ready solution for improving patient care and enhancing the efficiency of remote healthcare.

CHAPTER 2
TECHNOLOGY IN GENERAL

In today's fast-paced world, healthcare is shifting. More and more, we're moving away from hospital visits and toward remote, real-time care that meets people where they are—whether at home, at work, or on the go. That's exactly where **Vital Guard** comes in. This smart system is designed to monitor a person's vital health signs continuously and wirelessly, using a combination of compact devices, cloud technology, and an intuitive online dashboard. At the heart of Vital Guard is a tiny but powerful device called the **ESP32**. It might not look like much, but it's doing a lot behind the scenes. Think of it as the brain of the operation. It connects to several biomedical sensors, gathers health data in real time, and sends it instantly to the cloud—no wires, no fuss. Thanks to its built-in Wi-Fi and Bluetooth, the ESP32 can communicate smoothly and efficiently, and it barely uses any power, making it perfect for wearable or portable health devices that need to keep running for long periods.

Vital Guard is built to keep tabs on the vital signs that matter most—heart rate, oxygen levels, body temperature, and even heart rhythms. It does this using a set of small, specialized sensors. The **MAX30102** sensor measures heart rate and blood oxygen saturation (SpO_2). It uses light to "see" changes in your blood, which might sound like sci-fi but is actually a proven, accurate method. It is super compact, reliable, and works well even in tricky environments—like if you're moving around or in a low-light setting. For a deeper look at heart health, Vital Guard includes an **AD8232 ECG sensor**, which records the electrical activity of your heart. This is how doctors can spot irregular heart rhythms or early warning signs of more serious conditions. To keep track of body temperature, the system uses a **DHT11** sensor. While it's simpler than the other sensors, it does the job well—offering quick, real-time readings that help round out the overall health picture.

All of this data flows through the ESP32 and gets sent straight to the cloud—in this case, **Firebase**, a powerful backend service that keeps everything organized and up to date. Firebase is like the digital headquarters for VitalGuard. It stores all the health data, keeps it synced across devices, and makes sure that updates happen in real time. So if a patient's heart rate suddenly spikes or their oxygen levels drop, that information shows up immediately for doctors or caregivers—no delay, no missed signals. Firebase is secure and built to scale. Whether you're tracking one patient or hundreds, it ensures everyone gets the care and attention they need, without overwhelming the system.

The user interface, the part people actually see and interact with is designed to be clean, responsive, and easy to under- stand. It's built with JavaScript and **React**, a modern framework that helps make everything feel snappy and smooth. From the dashboard, doctors or caregivers can see live graphs, receive alerts, and monitor key vitals at a glance. If something seems off, they can act immediately. And since the dashboard is web-based, it can be accessed from any phone, tablet, or computer—anywhere there's internet.One of the best things about Vital Guard is that it can be upgraded according to latest technologies. Because it's built with widely available components and open technologies, it's easy to upgrade. We can add new sensors and can integrate it to new systems. Vital Guard can be customized for different settings, whether it's home monitoring for chronic conditions, recovery tracking after surgery, or healthcare support in remote areas. Its flexible and ready for the future of healthcare demands.

It's a complete health monitoring system that brings medical care into the modern age. By combining small, smart sensors, real-time cloud communication, and a user-friendly interface, it helps ensure that patients get the care they need, when they need it—without needing to be in a hospital room. It's efficient, accessible, and built for the future of healthcare—where monitoring doesn't stop when the patient leaves the clinic, and peace of mind is always within reach. It's efficient, accessible, and built for the future of healthcare—where monitoring doesn't stop when the patient leaves the clinic, and peace of mind is always within reach.

Around the world, millions of people don't have regular access to doctors, especially in remote areas. According to the World Health Organization, more than 60% of people globally live in places where healthcare isn't easy to get. For them, some- thing like Vital Guard could make all the difference—it brings reliable, real-time monitoring into the home, using just a simple internet connection and affordable hardware. And it's not just about access. It's about prevention. Studies have shown that remote monitoring can cut down hospital readmissions by nearly 40%**.** Considering someone with a heart condition who normally needs frequent check-ups, with Vital Guard, their heart rhythm, oxygen levels, and temperature are being watched constantly. It's also incredibly helpful in medical recovery. In case of a person who just had surgery and is sent home to rest. Usually, they'd just hope they're healing okay until their follow-up appointment. But with Vital Guard, their vitals are being monitored day and night. If a fever starts or their heart rate spikes, their doctor is notified in real time. That kind of immediate feedback can stop a small issue from turning into a medical emergency.

The technology inside Vital Guard—the ESP32 chip and sensors like the MAX30102 and DHT11—are all low-cost and widely available. This makes it accessible to common man and make it suitable to be purchased in low expense. Hence, schools, clinics, families, and even nonprofit organizations could use on a large scale. Vital Guard can adapt itself with the latest technologies. Over time, it could get smarter with AI features that predict problems before they happen, or connect with GPS for patients who are out and about. It can even be expanded to track new types of health data. It's flexible, upgradable, and ready for whatever comes next.

CHAPTER 3

TECHNOLOGY IN SPECIFIC

3.1 BLOCK DIAGRAM

At the core of this setup is the **ESP32 microcontroller**, a small but powerful component that acts like the system's brain. Connected to it are a set of biomedical sensors, each with a specific job: one keeps track of the heart rate, another monitors the electrical activity of the heart (ECG), and yet another checks the body's temperature. These sensors are constantly collecting valuable data about the person wearing or using the device—quietly and continuously, without the need for any action from the user.

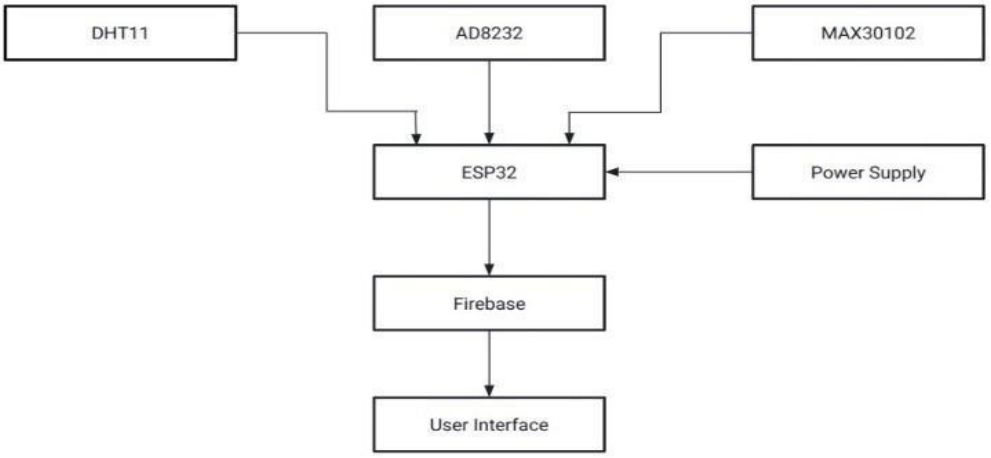


Fig 3.1 Block Diagram

The block diagram shown in Fig 3.1 gives a clear visual overview of how the Vital Guard system works behind the scenes. It represents a designed system that records someone's health in real time. It immediately sends the information wirelessly to Firebase, a secure, cloud-based platform where the data is stored and kept up-to-date in real time. Firebase can be considered as system's memory and communication hub as it makes sure the data is available to the right people.

Finally, that health data is displayed on a user interface which is a simple, interactive dashboard that can be accessed from a phone, tablet, or computer. Vital Stats can be accessed using this dashboard. DHT11 (Temperature Sensor. To record body temperature, the Vital Guard system uses a small but reliable sensor known as the **DHT11**. It plays an important role in painting a full picture of a person's health. This sensor constantly checks the user's body temperature and sends that information directly to the **ESP32 microcontroller**, which acts like the system's central processor. Just like how a thermometer helps you know when you might have a fever, the DHT11 works behind the scenes, checking for even the slightest changes in temperature. Temperature is a crucial parameter through which we can indicate if something wrong occur in our body. If it rises unexpectedly, it could indicate the start of an infection or inflammation—and early warning means faster action. Due to its connection with the ESP32, the DHT11 doesn't just record the temperature but transmits it for storing in cloud. Within seconds, that data is transmitted wirelessly to the cloud, where doctors or caregivers can see it in real time. So while the DHT11 might be just one part of the Vital Guard system, its role is vital—helping to catch health issues early, support recovery, and give both patients and caregivers a little more peace of mind.

One of the most important parts of the Vital Guard system is the **AD8232 sensor**, a compact and energy-efficient device specially designed to monitor the heart's electrical activity—which is referred as an **ECG** or **electrocardiogram**. It's small and runs on very little power, but this sensor plays a big role in keeping an eye on heart health. Every heartbeat sends out a tiny electrical signal, and the AD8232 is smart enough to pick up on these signals with surprising accuracy. It analyses the pattern of heartbeat. By tracking these signals, it helps detect any irregularities—such as missed beats, abnormal rhythms, or early signs of heart- related issues that might otherwise go unnoticed. Once the AD8232 picks up the ECG data, it immediately passes the information on to the **ESP32 microcontroller**, which then takes over processing and transmission. From there, the data gets sent to the cloud so it can be viewed in real time by doctors, nurses, or caregivers anywhere in the world. This seamless flow of information means that heart health can be monitored closely without the need for bulky hospital equipment or constant checkups.

To keep track of both heartrate and blood oxygen levels, the Vital Guard system uses a sensor called MAX30102. It has a tiny red LED, which emits light into our skin and analyze blood flowing through our capillaries. By analyzing how the light is observed and reflected, the sensor can figure out how fast our heart is beating and how much oxygen our blood is carrying. With every heartbeat, the amount of light observed changes lightly and the sensor picks up on These subtle shifts. It's compact, low power and built right into a portable device. Once the MAX30102 gathers the required information, it converts the raw readings into digital signals and sends them straight to ESP32 Microcontroller.

3.2 ESP32 (Microcontroller)

At the heart of the **Vital Guard system** is the **ESP32 microcontroller**—an incredibly powerful chip that serves as the system's central brain and communication hub. Even though it has a small size, it plays a huge role in making everything work seamlessly behind the scenes. The ESP32 is responsible for **gathering information from all three sensors** in the system. It listens to the **DHT11**, which tracks body temperature, the **AD8232**, which monitors the heart's electrical signals (ECG), and the **MAX30102**, which keeps tabs on heart rate and blood oxygen levels. After collecting and organizing the data, ESP32 sends it wirelessly over Wi-Fi to Firebase, a secure cloud database where the information is stored and shared. This allows doctors, nurses, and other professionals to view the health data in real time from any device, irrespective of their locations across the globe.

.

Another essential job the ESP32 handles is managing power. Since the Vital Guard system is meant to be **portable and wearable**, it's crucial that it can run for long periods without constantly needing to be recharged. The ESP32 is designed to be energy-efficient, carefully bal ancing performance and battery life. This means patients can wear the device irrespective of time and place. In concise, ESP32 can be considered as a very important component of Vital Guard system keeping the flow of information smooth, timely, and efficient. Without it, the sensors would have no coordination, the data would go nowhere, and the idea of real-time health monitoring simply wouldn't be possible.

3.2.1 Power Supply

Every smart system needs a reliable source of energy to keep it running, and in the case of Vital Guard, that job falls to the **power supply block**. While it may not be as flashy as the sensors or the ESP32 microcontroller, it's essential as it supplies power that help in the functioning of the Vital Guard system. This block is responsible for delivering just the right amount of **electrical power** to the entire system—including the **ESP32** and all the connected sensors. Whether it's the temperature readings from the DHT11, the ECG signals from the AD8232, or the heart rate and oxygen levels detected by the MAX30102, none of it would be possible without stable and consistent power. One of the most thoughtful aspects of the Vital Guard design is its **low power consumption**. All components were chosen not just for their performance, but for how much energy they re- quire to function effectively. This makes the entire system incredibly efficient—an important feature when the device is used as a **wearable or portable health monitor**. If someone is wearing Vital Guard throughout the day while going about their routine, or even while sleeping, due to the system's energy-efficient design, it can keep monitoring vitals continuously without draining the battery too quickly. This means fewer interruptions for recharging, more freedom for the user, and a smoother, more reliable health-tracking experience. In essence, this power block ensures that the Vital Guard system stays awake and alert— quietly enabling 24/7 monitoring and peace of mind. It might not be the most visible part of the system, but without it, none of the other technology could come to life.

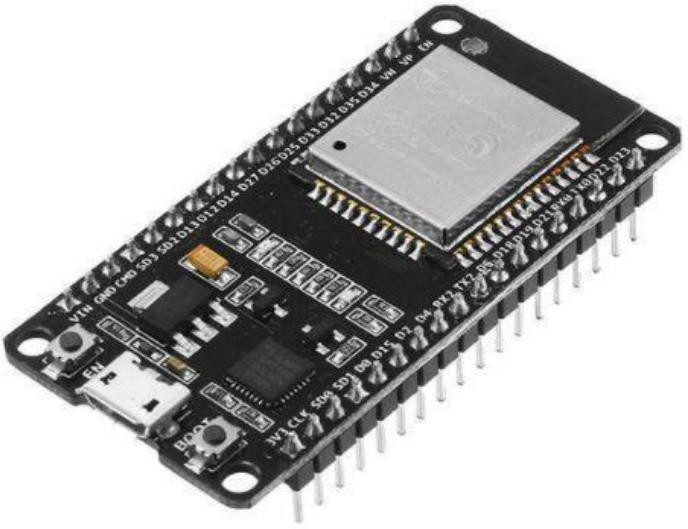


Fig 3.2.1 ESP32 image

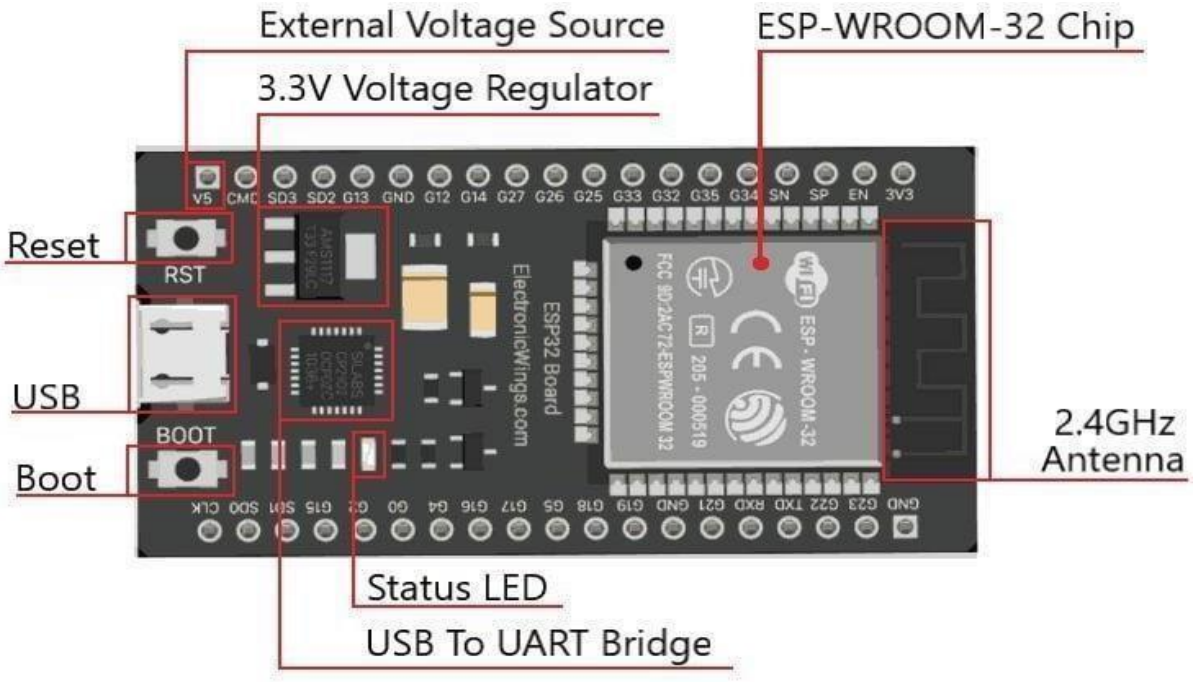


Fig 3.2.2 ESP32 pin configuration image.

3.3 SENSORS USED

3.3.1 MAX30102

The MAX30102 is a sophisticated biosensor module that combines pulse oximetry and heart rate monitoring into a single, compact device. It's like a small but powerful health tracker that's capable of measuring two vital signs—heart rate and blood oxygen levels (SpO_2)— with a surprising level of accuracy. At its core, the MAX30102 is equipped with internal LEDs, photodetectors, optical elements, and low-noise electronics. These elements work together to shine light through the skin and measure how the blood absorbs that light. The LEDs emit red and infrared light, while the photodetectors capture the light that is reflected back. By analyzing how the light interacts with the blood vessels, the MAX30102 can determine both the heart rate and blood oxygen levels in real-time. One of the standout features of the MAX30102 is its ability to reject ambient light. This is crucial because, in a real-world setting, there's always background light that can interfere with measurements. Irrespective if being in indoors or outdoors, the MAX30102's design ensure that it only focuses on the data that matters, providing accurate readings.

The importance of MAX30102 lies not just in its functionality but also in its integration. It's designed as an all-in-one solution, meaning it brings together all the necessary components for pulse oximetry and heart-rate monitoring into one convenient module. This makes it easier for developers to incorporate the MAX30102 into mobile or wearable devices, whether it's a fitness tracker, a health monitor, or a medical device. By providing a complete system solu- tion, the MAX30102 simplifies the design and development process, making it easier to cre- ate products that can monitor key health metrics with accuracy and reliability. Overall, the MAX30102 is a compact but powerful tool for tracking two of the most im- portant indicators of health—heart rate and blood oxygen levels—and it does so with remark- able precision. It's a key component in making wearable health technology more accessible, practical, and reliable.

The importance of MAX30102 lies not just in its functionality but also in its integration. It's designed as an all-in-one solution, meaning it brings together all the necessary components for pulse oximetry and heart-rate monitoring into one convenient module. This makes it easier for developers to incorporate the MAX30102 into mobile or wearable devices, whether it's a fitness tracker, a health monitor, or a medical device. By providing a complete system solu- tion, the MAX30102 simplifies the design and development process, making it easier to cre- ate products that can monitor key health metrics with accuracy and reliability.

Overall, the MAX30102 is a compact but powerful tool for tracking two of the most important indicators of health—heart rate and blood oxygen levels—and it does so with remark- able precision. It's a key component in making wearable health technology more accessible, practical, and reliable.

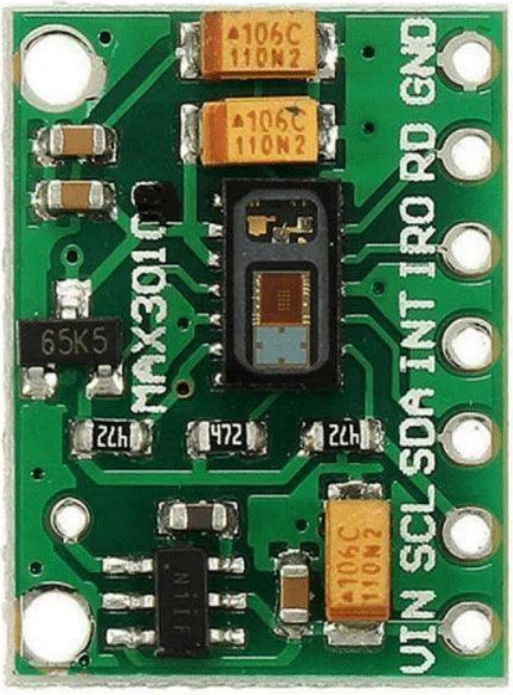


Fig 3.3.1 MAX30102

3.3.2 AD8232

The AD8232 is a small but incredibly important component used to measure the electrical signals generated by the heart, specifically for ECG (electrocardiogram) monitoring. It is a signal booster for the heart's electrical activity, which, in its natural state, is very weak and hard to detect without help. When the heart beats, it generates electrical signals that travel through the body. These signals are what doctors use to assess the health of the heart by looking at the ECG. However, these signals are so small that they need to be amplified before they can be analyzed or recorded by a medical device. This is where the AD8232 comes in. The AD8232 works by amplifying those small heart signals, making them strong enough to be captured and processed by other devices like the ESP32 in the VitalGuard system. It's like having a microphone that picks up faint sounds and turns them into something loud and clear. By strengthening these signals, the AD8232 ensures that the heart's electrical activity can be accurately measured, displayed, and analyzed in real time.

One of the key features of the AD8232 is its signal conditioning ability. It doesn't just amplify the signals; it also ensures that they are clean and free of noise or interference. This is important because heart signals can sometimes be contaminated by other electrical activity in the body (like muscle movements or electrical noise from nearby devices). The AD8232 filters out these unwanted signals, allowing only the heart's true electrical activity to come through, making it easier for healthcare providers to accurately read and interpret the ECG. In simple terms, the AD8232 is the heart's translator, turning its weak electrical signals into clear, readable data. Without it, devices wouldn't be able to track heart rhythms or detect abnormalities, which are critical for diagnosing heart problems. The AD8232 makes this process possible in a compact, low-power format, allowing wearable devices and portable health monitors to provide valuable insight into a person's heart health.

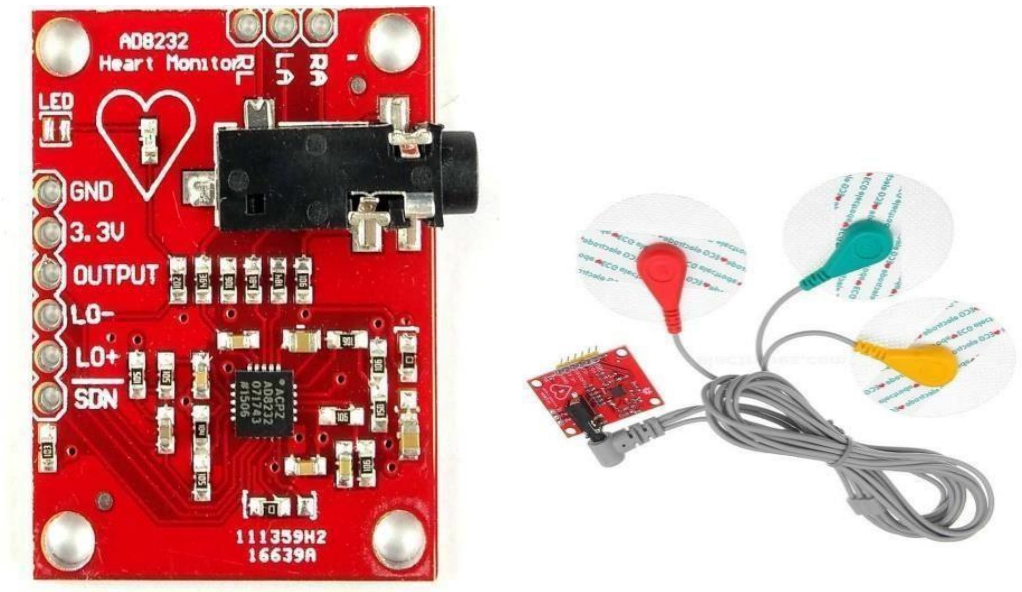


Fig 3.3.2 AD8232 & ECG Module

3.3.3 DHT11

The DHT11 sensor is a small but incredibly useful device that measures temperature and humidity—two key factors that can tell us a lot about our environment and our health. The DHT11 uses two main components to gather this information—a thermistor and a capacitive humidity sensor. A thermistor is a special type of resistor that changes its resistance based on temperature. So, when the temperature goes up or down, the thermistor adjusts how much it resists the flow of electrical current. The colder it gets, the higher the resistance, and the warmer it gets, the lower the resistance. This change is a very accurate way of measuring temperature, and the sensor uses it to calculate the exact temperature of the surrounding air.Once the thermistor sensor gather their data, the DHT11 doesn't just keep it to itself. It converts all the temperature information into a digital signal, which is easy for computers and microcontrollers to understand. The data is then sent through a single- wire serial interface to a microcontroller, like the ESP32 in the VitalGuard system. This makes it simple for the system to read and use the information, which could then be used for monitoring a patient's environment or other applications where accurate environmental data is essential.

Once the thermistor sensor gather their data, the DHT11 doesn't just keep it to itself. It converts all the temperature information into a digital signal, which is easy for computers and microcontrollers to understand. The data is then sent through a single- wire serial interface to a microcontroller, like the ESP32 in the VitalGuard system. This makes it simple for the system to read and use the information, which could then be used for monitoring a patient's environment or other applications where accurate environmental data is essential.

In summary, the DHT11 is like a little environmental detective, constantly measuring the temperature and humidity around it and sending that data to the system so it can be used in real-time. Whether it's tracking how a room's temperature might be affecting a patient or en- suring that a wearable device is operating in an ideal environment, the DHT11 provides the essential data to make sure everything is in balance.

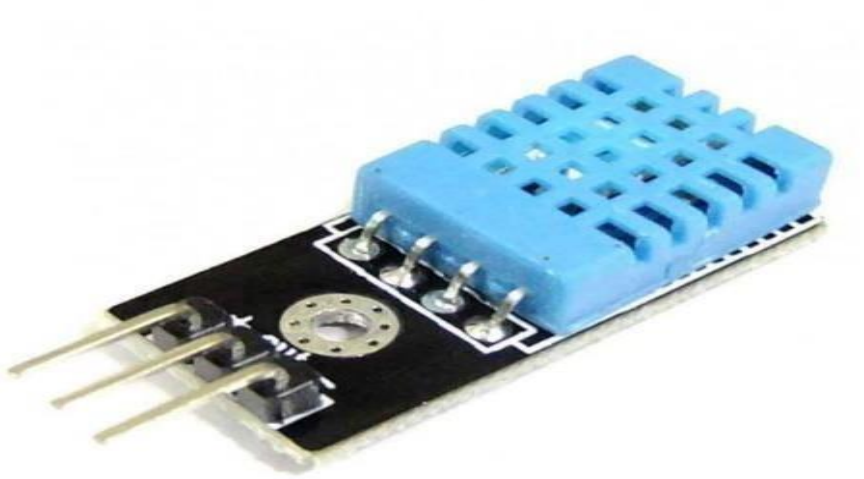


Fig 3.3.3 DHT11

CHAPTER 4

HARDWARE IMPLEMENTATION

4.1 Circuit Diagram

Fig 4.1 shows the connections between hardware components in the model.

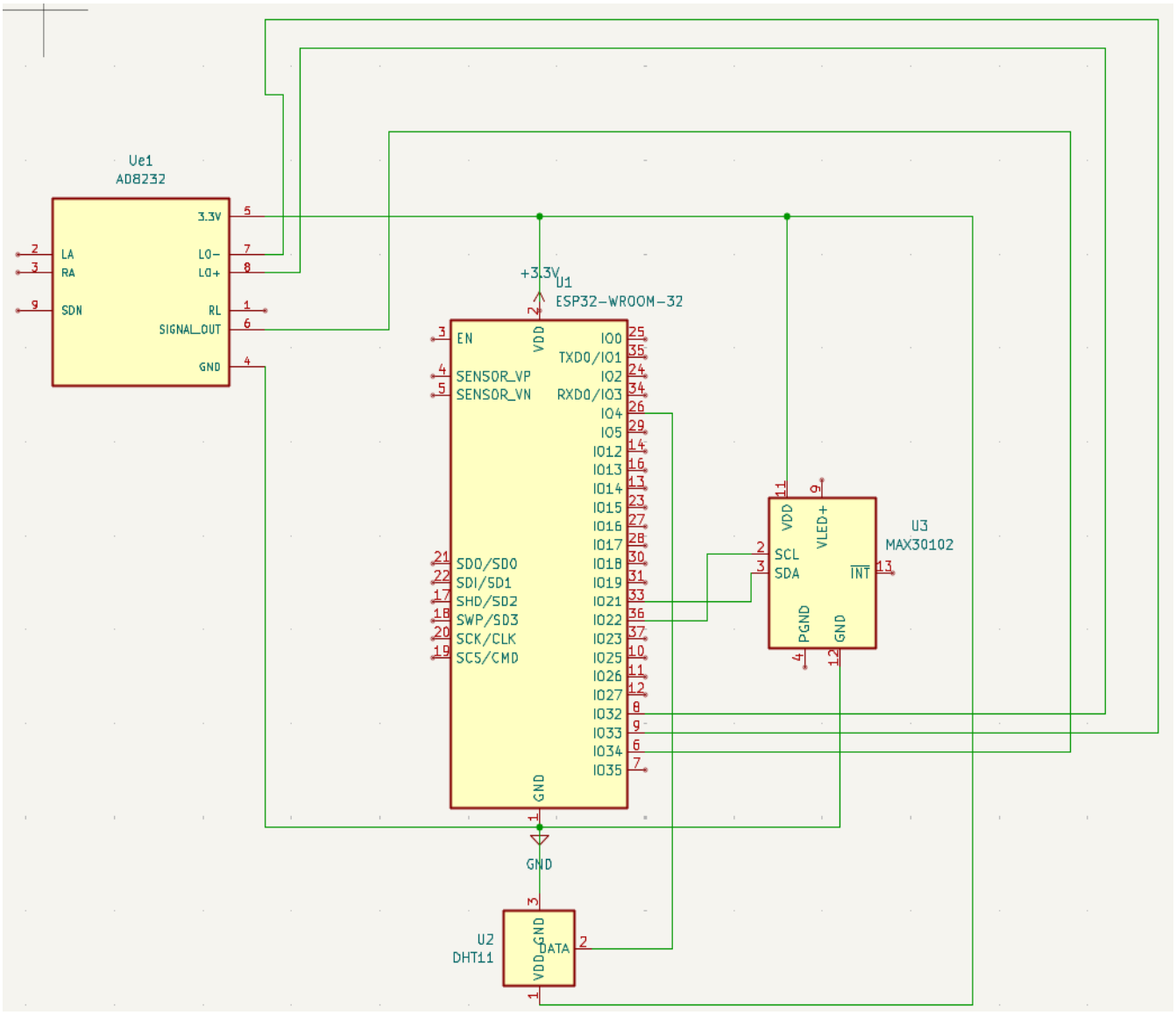


Fig 4.1 Circuit Diagram

4.2 WORKING

4.2.1 Power Supply

The ESP32 is powered with the help of USB from a laptop, providing a 5V input. This is regulated to 3.3V by the ESP32's onboard voltage regulator. It steps down the 5V USB input to 3.3V. All three sensors (AD8232, MAX30102, DHT11) are powered through the 3.3V pin of the ESP32. A common GND is shared between ESP32 and all sensors to maintain proper voltage references.

4.2.2 ESP32-WROOM-32

The ESP32 microcontroller serves as the intelligent core of the Vital Guard system, responsible for reading, processing, and managing data from all connected sensors. It communicates with these sensors through its various input/output (I/O) pins, which include both digital and analog pins. These pins allow the ESP32 to interact with a range of sensor types—whether it's receiving digital signals from the MAX30102 pulse oximeter, interpreting analog ECG signals from the AD8232, or handling temperature readings from the DHT

4.2.3 MAX30102(Heart Rate & SpO_2 Sensor)

MAX30102 communication with ESP32

MAX30102 Pin	ESP32 Pin	Function
SDA	GPIO 22	I2C
SCL	GPIO 21	I2C
INT	Not Connected	Optional
VCC	3.3V	Power
GND	GND	Ground

Table 1.1 MAX30102 pin configuration with ESP32

In the Vital Guard system, communication between the ESP32 microcontroller and certain sensors—particularly the MAX30102 pulse oximeter—is established through the I²C (Inter-Integrated Circuit) interface, a widely used serial communication protocol. On the ESP32 board, GPIO21 functions as the SDA (Serial Data Line) and GPIO22 serves as the SCL (Serial Clock Line). These pins enable seamless data exchange between the microcontroller and the sensor using just two wires, which is ideal for keeping the system compact and easy to wire. One of the key benefits of using I²C is its simplicity—the majority of modern sensor libraries are already designed to support I²C communication. This significantly reduces the amount of code configuration required, allowing developers to focus more on functionality and data processing rather than low-level communication details.

The MAX30102 sensor, connected via I²C, is responsible for continuously measuring pulse rate and blood oxygen saturation (SpO_2)—two vital health parameters that reflect both cardiovascular and respiratory function. It uses a combination of red and infrared LEDs to shine light into the skin, typically through a fingertip. A built-in photodetector then measures how much light is absorbed by the blood. This absorption fluctuates with each heartbeat, and the amount of light absorbed by oxygen-rich versus oxygen-poor blood differs depending on the wavelength of light used. By analyzing these changes, the MAX30102 can determine both heart rate and oxygen saturation with a high level of accuracy. What makes this sensor especially efficient is its internal Analog-to-Digital Converter (ADC), which processes the analog optical signals internally and sends clean, digitized data directly to the ESP32 through the I²C interface. This offloads some of the processing from the microcontroller and ensures faster, more accurate readings. Due to its design and efficient communication setup, the MAX30102 plays a crucial role in delivering real-time health insights in a format that's both precise and easy to manage for the system.

4.2.4 AD8232 (ECG Sensor)

AD8232 Connection with ESP32

AD8232 Pin	ESP32 Pin	Function
SIGNAL	GPIO34	Analog Output
LO+	GPIO32	Lead-Off Detection
LO−	GPIO33	Lead-Off Detection
3.3V	3.3V	Power Supply
GND	GND	Ground

Table 1.2 AD8232pin configuration with ESP32

As shown in Table 1.2, the ESP32 microcontroller is interfaced with the ECG sensor module (AD8232) through specific GPIO pins to ensure accurate and reliable cardiac monitoring. GPIO34, one of the ESP32's ADC (Analog-to-Digital Converter) pins, is used to read the analog ECG signals generated by the sensor. This makes it ideal for capturing the subtle electrical activity of the heart and converting it into a digital format that can be visualized and analyzed. The AD8232 continuously outputs a smooth analog waveform, representing the heart's electrical impulses in real time. This waveform can be plotted directly in the Arduino IDE during development or transmitted to the cloud for remote cardiac monitoring, where healthcare professionals can access it instantly through the web interface. To further enhance the reliability of the ECG readings, GPIO33 and GPIO32 are utilized to monitor the lead-off status. This feature helps detect whether the ECG electrodes are properly attached to the patient's skin. If an electrode becomes loose or disconnected, the system can instantly identify the issue and alert the user or caregiver. This lead-off detection is a critical part of ensuring that the system delivers consistent and trustworthy readings, which is espe- cially important in real-time health monitoring scenarios where accuracy can directly impact clinical decisions. Together, these connections and features allow the ESP32 to act as a smart, responsive hub for continuous and remote heart health observation.

4.2.5 DHT11 (Temperature Sensor)

AD8232 connection with ESP32

DHT11 Pin	ESP32 Pin	Function
DATA	GPIO 4	Output Data
VCC	3.3V	Power supply
GND	GND	Ground

Table 1.3DHT11 pin configuration with ESP32

4.3 LIST OF COMPONENTS

4.3.1 ESP32

At the heart of the Vital Guard system is a **powerful microcontroller** that does more than just process data—it acts as the brain of the entire operation. The **ESP32**, known for its built- in **Wi-Fi and Bluetooth connectivity**, brings wireless communication capabilities to the system, making remote monitoring possible without the need for additional modules. It features a **dual-core 32-bit processor**, giving it the muscle to handle multiple tasks simultaneously—like reading sensor inputs, processing data, and sending updates to the cloud—all in real time. What really makes the ESP32 versatile is its wide array of **GPIO (General Purpose Input/Output) pins**, which allow it to easily interface with various sensors used in the project, such as the heart rate monitor, ECG sensor, and temperature sensor. This makes it not only a powerful processor but also a flexible controller that can adapt to different configurations and healthcare needs. Despite its advanced features, the ESP32 is compact and energy-efficient, making it ideal for wearable or portable health monitoring applications where space and battery life are crucial.

4.3.2 MAX30102

One of the key components in the Vital Guard system is an integrated sensor module that handles both pulse oximetry and heart rate monitoring—the MAX30102. This compact yet highly capable module uses I²C communication, which allows it to easily connect with the ESP32 microcontroller using just a few wires, simplifying the overall design. Inside this tiny package are some impressive features: it comes with built-in red and infrared (IR) LEDs**,** along with **a** high-sensitivity photodetector. These components work together using a technique called photoplethysmography (PPG)**,** which measures the changes in light absorption as blood pulses through the fingertip. The red and IR light helps the sensor estimate both heart rate **and** blood oxygen saturation (SpO_2)—two vital parameters for assessing a person's cardiovascular and respiratory health. Despite its advanced functionality, the MAX30102 is energy-efficient and small in size, making it ideal for wearable health tech. It delivers precise, real-time data even in low-light or motion-prone environments, ensuring reliable performance in day-to-day monitoring scenarios.

4.3.3 DHT11

The Vital Guard system uses theDHT11 digital temperature sensor to keep track of a patient's body temperature—an essential health indicator, especially when monitoring for signs of fever or infection. DHT11 is user-friendly due to its single-wire communication protocol**,** which allows it to send temperature data to the ESP32 microcontroller using just one data line. This not only simplifies the wiring but also helps reduce power consumption—making it ideal for compact, portable health devices. The sensor provides temperature readings with an accuracy of around **±2°C**, which is sufficient for general health monitoring applications. Internally, the DHT11 uses a thermistor, which changes its resistance based on the surrounding temperature, and then converts that data into a digital signal. Its compact design, low cost, and ease of integration make it a practical choice for continuous temperature tracking, especially in remote or home-care settings where simple yet reliable monitoring tools are essential.

4.3.4 AD8232

To monitor the electrical activity of the heart, the Vital Guard system incorporates the AD8232 ECG module**—a** specialized analog front-end designed specifically for capturing biopotential signals such as heartbeats. The heart generates tiny electrical impulses with every beat, but these signals are often too weak and noisy to be processed directly by a microcontroller. AD8232 extracts**,** amplifies**,** and filters these delicate signals, making them clean and strong enough to be analyzed and interpreted. This process is crucial for generating accurate ECG (electrocardiogram) readings, which can reveal important insights into the heart's rhythm and overall function. Whether it's identifying irregular patterns or simply monitoring heart health over time, the AD8232 ensures the ECG data is both reliable and ready for real-time transmission. Its compact design and low power consumption make it ideal for wearable health devices and continuous remote monitoring applications.

CHAPTER 5

SOFTWARE IMPLEMENTATION

To bring the Vital Guard system to life, a thoughtful selection of development tools and platforms was made—each chosen not just for its functionality, but for how well it supports seamless integration, real-time communication, and user accessibility. The Arduino IDE was used for programming the ESP32 microcontroller for the following reasons. It offers a simple and intuitive interface that makes it easy to write, upload, and debug code. Irrespective of being a beginner or an experienced developer, the Arduino environment is approachable and efficient. One of its major strengths is its wide hardware compatibility, particularly with microcontrollers like the ESP32. Furthermore, it has a vast ecosystem of pre- built libraries—including those for sensors like the DHT11, AD8232, and MAX30102— making development faster and far less complicated. These libraries abstract away much of the lower-level code, allowing the focus to remain on system logic and integration rather than device-specific intricacies.

To handle the system's cloud-based data storage and real-time syncing, Firebase Real-time Database was selected. Its real-time update capabilities are crucial for a health monitoring system where even a few seconds can matter. As soon as the ESP32 sends new data— whether it's rise in temperature or a change in heart rhythm—Firebase ensures that this information is immediately reflected on the user interface. Being a cloud-hosted NoSQL database, it also eliminates the need for setting up and maintaining a physical backend server. Firebase's smooth integration with both microcontrollers and modern web frameworks makes it an ideal fit for projects that demand live data flow and quick accessibility from multiple devices.On the frontend, React was chosen to build the user interface that displays patient vitals in real- time. React is a component-based JavaScript library that makes building complex user interfaces both manageable and scalable. One of its standout features is efficient state man- agement, which allows the app to reflect real-time data updates without needing full page re- loads. This is especially important in a health monitoring context where live graphs, ECG waveforms, and status indicators need to change fluidly as new data comes in React's

modular structure also ensures that the application can be expanded or improved in the future with minimal hassle—whether that means adding new sensor visuals or refining user alerts.

Once the **ESP32** gathers important health data from the sensors—whether it's heart rate, body temperature, or ECG signals—it doesn't just keep it to itself. Instead, it sends that information to **Firebase**, which acts as the **real-time cloud backend** for the system. Firebase can be considered as a virtual storage space that lives in the cloud, constantly updating and syncing the data as it comes in. In the context of this project, **Firebase** plays a crucial role. In this, all of the health data is stored securely and in real time. As soon as the ESP32 sends the collected data, Firebase instantly processes it and stores it safely. This allows medical professionals, caregivers, or loved ones to access the information whenever they need it, without delay. If there's a sudden spike in the patient's heart rate or a drop in oxygen levels, the data is immediately sent to Firebase, where it's available for doctors to see on their device in just a few moments.

Firebase's real-time capabilities also ensure that this system works smoothly, even when multiple people are monitoring the same patient's health. Anyone viewing the dashboard can see the same up-to-date information instantly. This real-time syncing helps ensure that nothing slips through the cracks and that the patient receives the best possible care. Once the health data is sent to Firebase, it needs to be presented in a way that's easy to under- stand and act upon. That's where the user interface comes in—this is the visual dashboard that healthcare providers, caregivers, or even family members use to monitor a patient's health in real time. It can be considered as the control center for Vital Guard, where all the data comes together and is displayed in a clear, simple format. The user interface is built using JavaScript and React, which are powerful tools that make the dashboard interactive and responsive. In simpler terms, they help ensure that the interface updates quickly and smoothly, so the user always sees the most current data as it's collected. The interface shows important health stats like heart rate, an ECG waveform (which looks like a heartbeat rhythm), and the body temperature—all in easy-to-read visual formats. These are presented through live graphs, waveforms, and numerical displays that show exactly what's going on with the patient's health at that moment. Whether it's a steady heart rate, a fluctuating temperature, or the electrical signals of the heart, the data is shown in real-time.

The live display in the Vital Guard system can be an innovative technology for healthcare providers, especially when they're responsible for looking after multiple patients at once. If a nurse or doctor requires to keep track of several patients throughout the day, instead of having to go to each patient's room or wait for regular reports, they can check in remotely—whether they're at the clinic, working from their office, or even at home. The dashboard shows key health data, like heart rate, ECG patterns, and body temperature, all in one place, and it updates instantly. This means that healthcare providers can quickly see exactly how each patient is doing without any delay. This system helps healthcare providers stay ahead of problems. Instead of waiting for an emergency to happen—they can spot trends and patterns in the data. For example, if a patient's heart rate has been climbing steadily for a few days, the provider can notice it early and make adjustments before it becomes a serious issue. It's about detecting problems before they escalate, which can make all the difference in a patient's recovery. In short, the system is a way to make healthcare more personal, more efficient, and more accessible. By giving doctors and nurses the tools they need to monitor patients from afar, it helps ensure that patients stay safe, supported, and cared for—no matter where they are. Together, these tools—Arduino IDE, Firebase, and React—form a powerful development stack that supports every layer of the Vital Guard system, from data collection and transmission to real-time visualization and remote accessibility. Each technology contributes to the overall goal of the project: to create a reliable, user-friendly, and responsive health monitoring solution that brings critical medical data right to the fingertips of caregivers, doctors, and family members.

5.1 ALGORITHM

Program Algorithm

Step 1: Initialize the MAX30102 sensor and configure Wi-Fi and Firebase credentials within the ESP32 environment.

Step 2: Establish a connection to the local wireless network using predefined SSID and password parameters.

Step 3: Synchronize the device's internal clock using NTP servers to maintain accurate timestamps.

Step 4: Set up Firebase configuration by including API keys, database URLs, and authenticated user credentials.

Step 5: Verify Firebase readiness and implement error handling or reconnection logic if initialization fails.

Step 6: Continuously acquire raw IR and red light data from the MAX30102 sensor.

Step 7: Calculate the user's heart rate by analyzing time intervals between successive pulse peaks.

Step 8: Estimate the SpO2 percentage based on the ratio of red to IR light absorption.

Step 9: Simulate temperature data and read analog ECG voltage input for real-time signal tracking.

Step 10: Structure the acquired data into a JSON object along with a timestamp for identification.

Step 11: Transmit the JSON payload to Firebase Real-time Database under a unique MAC-

address-based path.

Step 12: Initialize Firebase services within the React frontend application using appropriate SDK methods.

Step 13: Subscribe to the Real-time Database path to receive live data updates using event-driven listeners.

Step 14: Extract and process the incoming JSON data to update component states for rendering.

Step 15: Present ECG signals and vital signs such as temperature, BPM, average BPM, and SpO2 through responsive UI elements and dynamic charts.

5.2 Steps and Procedure

1. Initialize Sensors.

2. Acquire Sensor Data.

3. Upload Data to Firebase.

4. Read Data from Firebase.

5. Display Data on Monitoring Interface.

CHAPTER 6

PCB LAYOUT

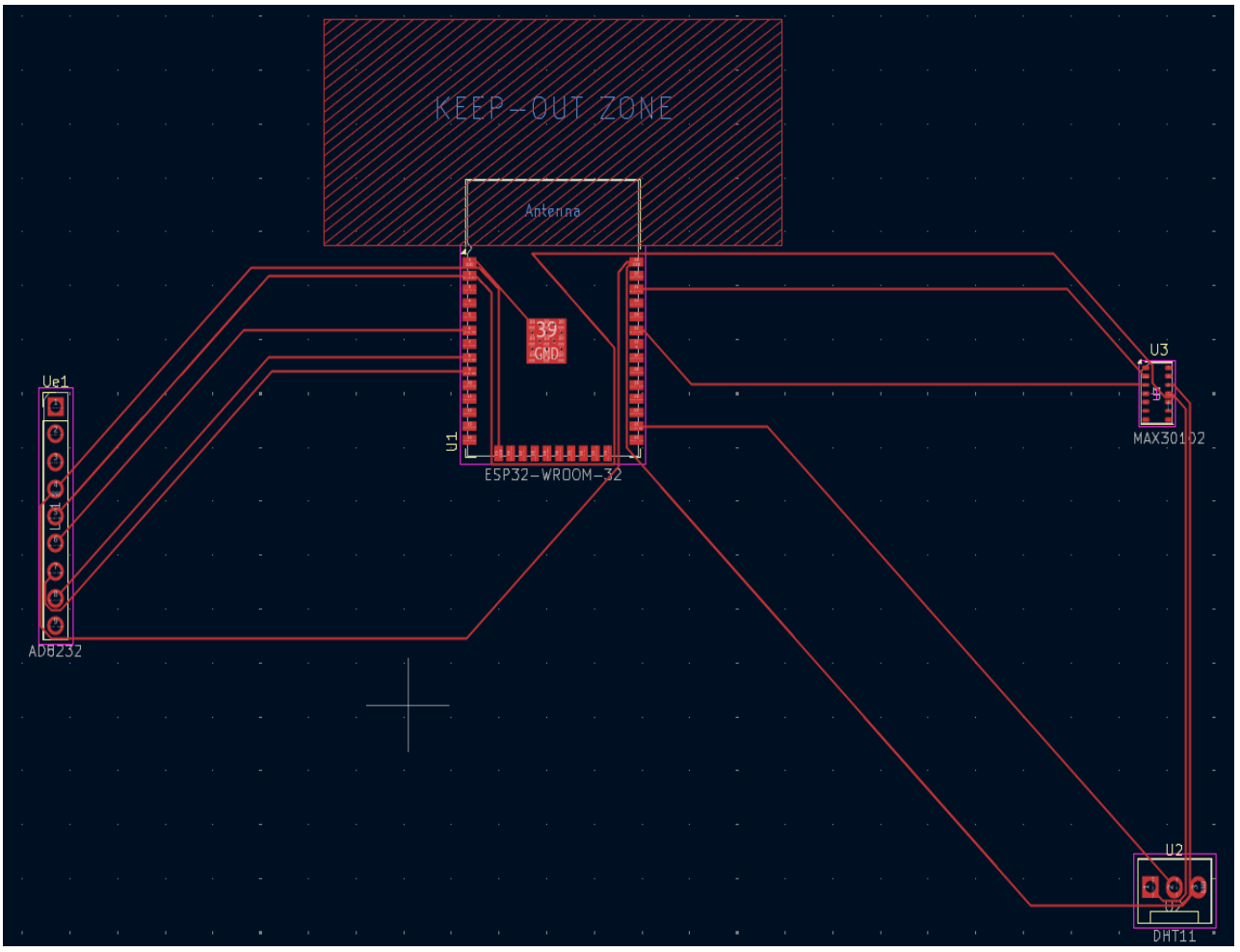


Fig 6.1 PCB Layout

The software used to design the PCB Layout of this Mini project was KiCad. KiCad is a free and open-source electronic design automation software suite used for creating electronic electronics circuit schematic and PCB Layout

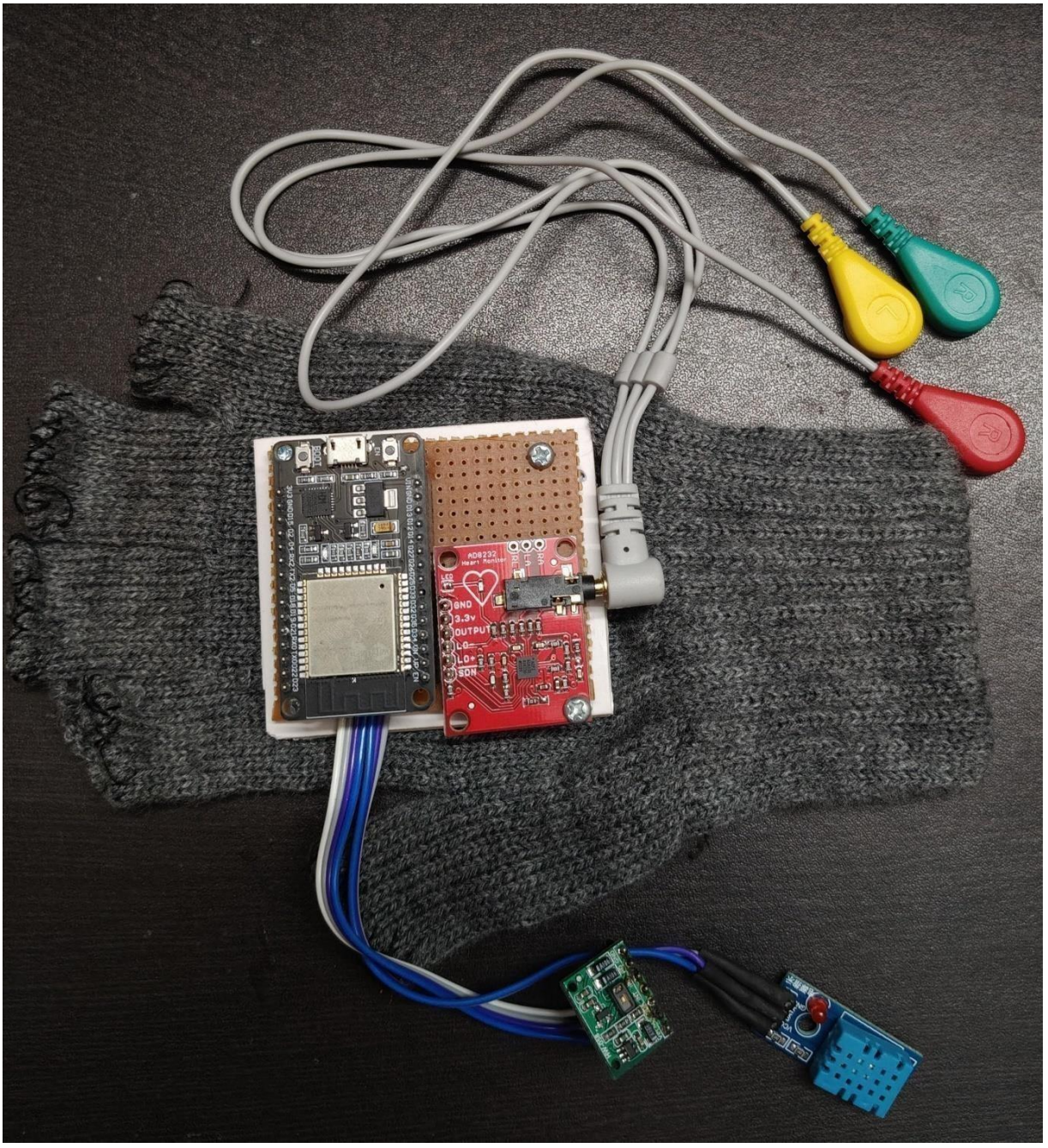


Fig 6.2 Working model

CHAPTER 7

RESULT

The ESP32 established a stable Wi-Fi connection and transmitted sensor data to Firebase with minimal latency. Vital parameters, including heart rate, SpO_2, temperature, and ECG voltage, were recorded and updated at regular intervals. The React-based frontend accurately displayed real-time data with smooth ECG waveform rendering. Overall, the system operated reliably during continuous testing, with consistent performance and minimal data loss. The below figure, Fig 7.3 shows the created dashboard using React which displays Temperature, SpO2, Heart Rate, Average BPM and ECG.

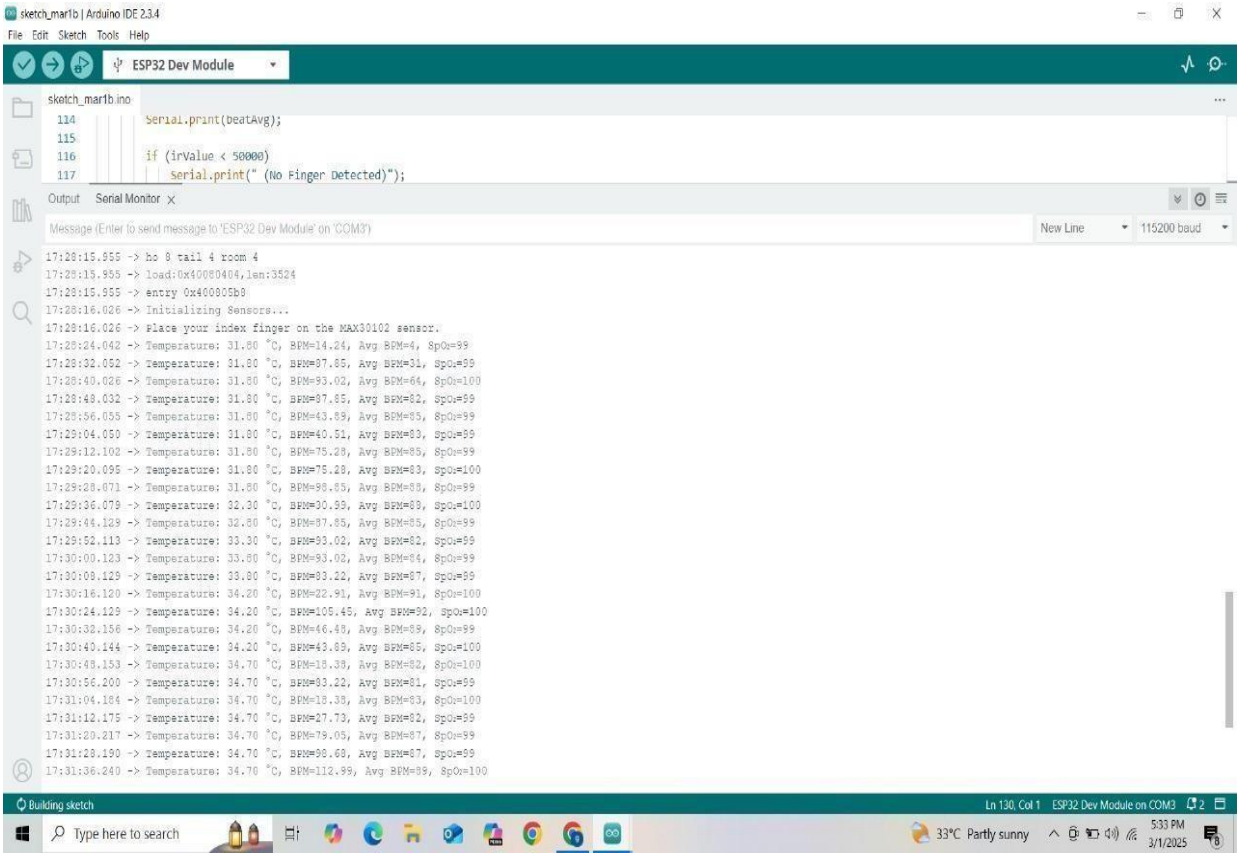


Fig 7.1 Heart rate, SPO2, Temperature

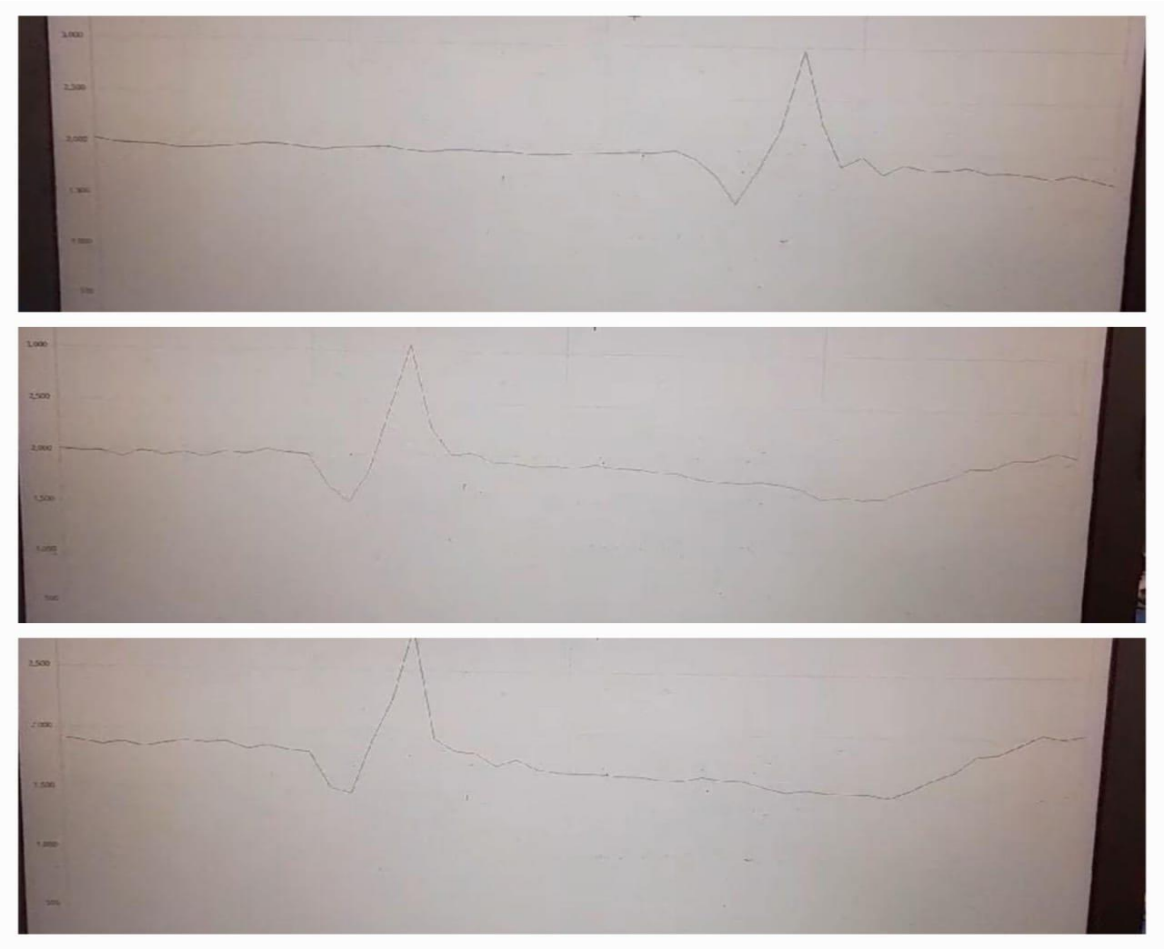


Fig 7.2 ECG in Serial Plotter

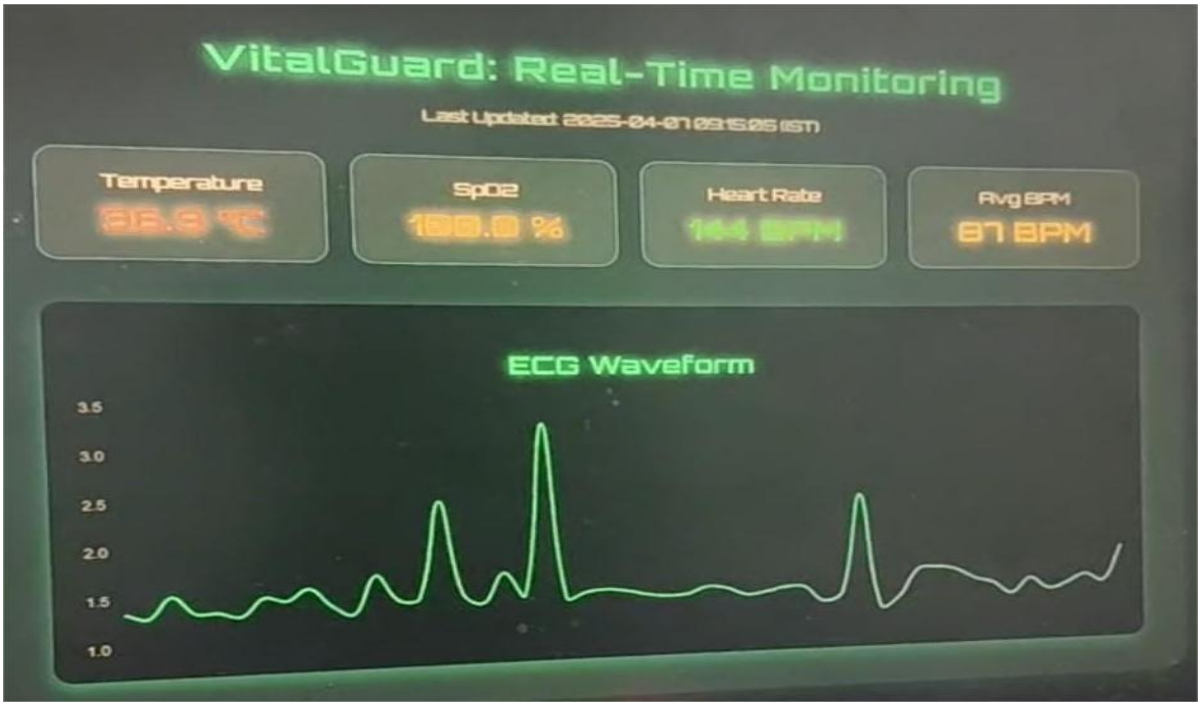


Fig 7.3 Display of real time using Frontend

CHAPTER 8

CONCLUSION

The Vital Guard system successfully integrates various sensors (MAX30102, AD8232, DHT11) with the ESP32 to monitor vital health data in real-time. Using Firebase for data storage and React for the frontend, the system enables seamless communication between the patient and healthcare providers. This approach enhances remote patient monitoring, ensuring efficient health tracking, faster response times, and improved patient care. With future enhancements, Vital Guard has the potential to revolutionize healthcare by offering scalable and accessible solutions for continuous monitoring.

REFERENCES

1. L. T. Tan, R. S. M. Shahrizal, M. N. M. Nadzri, N. Yahya and S. B. Hisham, "Remote Patient Monitoring System," *2022 IEEE 5th International Symposium in Robotics and Manufacturing Automation (ROMA)*, Malacca, Malaysia, 2022, pp. 1-6, doi: 10.1109/ROMA55875.2022.9915658

2. S. L. Deshmukh, F. Fernandes, S. Kulkarni, A. Patil, V. Jabade and J. Madake, "Patient Monitoring System," *2022 IEEE 2nd Mysore Sub Section International Conference (MysuruCon)*, Mysuru, India, 2022, pp. 1-6, doi: 10.1109/MysuruCon55714.2022.9972563.

3. S. Maqbool, M. Waseem Iqbal, M. Raza Naqvi, K. Sarmad Arif, M. Ahmed and M. Arif, "IoT Based Remote Patient Monitoring System," *2020 International Conference on Decision Aid Sciences and Application (DASA)*, Sakheer, Bahrain, 2020, pp. 1255-1260, doi: 10.1109/DASA51403.2020.9317213

4. T. Khan and M. K. Chattopadhyay, "Smart health monitoring system," *2017 International Conference on Information, Communication, Instrumentation and Control (ICICIC)*, Indore, India, 2017, pp. 1-6, doi: 10.1109/ICOMICON.2017.8279142.

APPENDIX

Program

ARDUINO CODE:

```
#include <WiFi.h>
#include <Firebase_ESP_Client.h>
#include <Wire.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include "MAX30105.h"
#include "heartRate.h"

// WiFi credentials
#define WIFI_SSID "IEDC Maker Studio"
#define WIFI_PASSWORD "Iedcmks@2023"

// Firebase credentials
#define API_KEY "AIzaSyBy2z2m-dAmEofUqdWP47WwGt3U56mYoc4"
#define DATABASE_URL "https://vital-guard-final-default-rtdb.asia-southeast1.firebasedatabase.app"

// Firebase objects
FirebaseDatafbdo;
FirebaseAuth auth;
FirebaseConfig config;
```

```
MAX30105 particleSensor;


// Heart rate variables
const byte RATE_SIZE = 4;
byte rates[RATE_SIZE];
byte rateSpot = 0;
long lastBeat = 0;
float beatsPerMinute = 0;
int beatAvg = 0;


unsigned long sendDataPrevMillis = 0;


void connectToWiFi() {
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
Serial.print("Connecting to Wi-Fi");
  unsigned long startTime = millis();
  while (WiFi.status() != WL_CONNECTED &&millis() - startTime< 15000) {
Serial.print(".");
delay(300);
  }
Serial.println();
  if (WiFi.status() == WL_CONNECTED) {
Serial.println("✅ Connected to WiFi");
Serial.print("IP: "); Serial.println(WiFi.localIP());
Serial.print("MAC: "); Serial.println(WiFi.macAddress());
  } else {
```

```
Serial.println("❌ Failed to connect to WiFi!");
  }
}

void setupTime() {
configTime(0, 0, "pool.ntp.org", "time.nist.gov");
time_t now = time(nullptr);
  while (now < 8 * 3600 * 2) {
delay(500);
Serial.print(".");
    now = time(nullptr);
  }
Serial.println("\n✅ Time synchronized");
}

void setup() {
Serial.begin(115200);
Serial.println("Setup started...");

connectToWiFi();
setupTime();

  // Initialize Firebase
config.api_key = API_KEY;
config.database_url = DATABASE_URL;
```

```
auth.user.email = "vitalguardmonitor@gmail.com";
auth.user.password = "VitalGuardS6EC";

  // Disable cert verification for testing
config.cert.data = nullptr;

Serial.println("Initializing Firebase...");
Firebase.begin(&config, &auth);
Firebase.reconnectWiFi(true);

  // Wait for Firebase to become ready
  unsigned long start = millis();
  while (!Firebase.ready() &&millis() - start < 15000) {
Serial.print(".");
delay(500);
  }

  if (Firebase.ready()) {
Serial.println("\n✅ Firebase initialized");
  } else {
Serial.println("\n❌ Firebase failed to initialize!");
Serial.println("Reason: " + fbdo.errorReason());
  }

  // Initialize MAX30105
```

```
  if (!particleSensor.begin(Wire, I2C_SPEED_STANDARD)) {
Serial.println(" ❌ MAX30102 not found. Using dummy values.");
  } else {
particleSensor.setup();
particleSensor.setPulseAmplitudeRed(0x0A);
particleSensor.setPulseAmplitudeGreen(0);
Serial.println(" ✅ MAX30102 ready. Place finger on sensor.");
  }
}

String getTimestamp() {
time_t now = time(nullptr);
char buf[20];
strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", localtime(&now));
  return String(buf);
}

void loop() {
  long irValue = particleSensor.getIR();
  long redValue = particleSensor.getRed();

  if (checkForBeat(irValue)) {
    long delta = millis() - lastBeat;
lastBeat = millis();
beatsPerMinute = 60 / (delta / 1000.0);
```

```
    if (beatsPerMinute> 20 &&beatsPerMinute< 255) {
      rates[rateSpot++] = (byte)beatsPerMinute;
rateSpot %= RATE_SIZE;
beatAvg = 0;
      for (byte i = 0; i< RATE_SIZE; i++) beatAvg += rates[i];
beatAvg /= RATE_SIZE;
    }
  }

  if (millis() - sendDataPrevMillis> 2000) {
sendDataPrevMillis = millis();

    float ratio = (redValue> 0 &&irValue> 0) ? (float)redValue / irValue : 0;
    float spo2 = 0;
    if (ratio > 0) {
      spo2 = 110 - 25 * ratio;
      spo2 = constrain(spo2, 0, 100);
    }

    float temperature = random(360, 380) / 10.0;
    float ecgVoltage = analogRead(34) * (3.3 / 4095.0);
    String timestamp = getTimestamp();

FirebaseJsonjsonData;
jsonData.set("timestamp", timestamp);
jsonData.set("temperature", temperature);
```

```
jsonData.set("ecg_voltage", ecgVoltage);
jsonData.set("bpm", beatsPerMinute);
jsonData.set("avg_bpm", beatAvg);
jsonData.set("spo2", spo2);

    String macAddress = WiFi.macAddress();
    String path = "/vital-data/" + macAddress;

    if (Firebase.ready()) {
Serial.println("📤 Uploading to Firebase...");
      if (Firebase.RTDB.setJSON(&fbdo, path.c_str(), &jsonData)) {
Serial.println("✅ Data uploaded!");
      } else {
Serial.print("❌ Upload failed: ");
Serial.println(fbdo.errorReason());
      }
    } else {
Serial.println("❌ Firebase not ready!");
    }
  }
}
```

JAVASCRIPT CODE:

```
import React, { useEffect, useState } from "react";
import { database } from "./firebase";
import { ref, onValue } from "firebase/database";
import { Line } from "react-chartjs-2";
import { Chart as ChartJS, LineElement, CategoryScale, LinearScale, PointElement, Tooltip } from "chart.js";
import './VitalSigns.css'; // Import the external CSS file

// Register Chart.js components
ChartJS.register(LineElement, CategoryScale, LinearScale, PointElement, Tooltip);

const VitalSigns = () => {
  const [data, setData] = useState({
    bpm: 0,
avg_bpm: 0,
    spo2: 0,
    temperature: 0,
ecg_voltage: [], // Array to store multiple ECG values for the graph
    timestamp: "",
    loading: true,
    error: null,
  });

useEffect(() => {
    // Use the MAC address from the ESP32
```

```
    const macAddress = "F0:24:F9:45:79:18"; // Replace with your device's MAC address
    const dataRef = ref(database, vital-data);


    const unsubscribe = onValue(dataRef, (snapshot) => {
      try {
        if (snapshot.exists()) {
          const newData = snapshot.val();
console.log("Firebase Data:", newData); // Debug Firebase data


          // Find the latest entry if the path contains multiple entries
          let latestData = newData;
          if (typeofnewData === 'object' && !Array.isArray(newData)) {
latestData = Object.values(newData).pop() || {};
          }


setData((prevState) => ({
            timestamp: latestData.timestamp || prevState.timestamp || "",
            temperature: latestData.temperature || prevState.temperature || 0,
            spo2: latestData.spo2 || prevState.spo2 || 0,
            bpm: latestData.bpm || prevState.bpm || 0,
avg_bpm: latestData.avg_bpm || prevState.avg_bpm || 0,
ecg_voltage: [...prevState.ecg_voltage.slice(-50), latestData.ecg_voltage || 0].slice(-50), // Keep last 50 ECG values
            loading: false,
            error: null,
          }));
```

```
        } else {
setData((prevState) => ({
            ...prevState,
            loading: false,
            error: "No data available at this path.",
          }));
        }
      } catch (error) {
console.error("Error fetching data:", error);
setData((prevState) => ({
          ...prevState,
          loading: false,
          error: "Error fetching data: " + error.message,
        }));
      }
    }, (error) => {
console.error("Firebase onValue error:", error);
setData((prevState) => ({
        ...prevState,
        loading: false,
        error: "Firebase error: " + error.message,
      }));
    });


    // Cleanup subscription on unmount
    return () =>unsubscribe();
```

```
}, []);


// ECG Graph Data
const ecgData = {
  labels: Array.from({ length: data.ecg_voltage.length }, (_, i) =>i + 1),
  datasets: [
    {
      label: "ECG Voltage (mV)",
      data: data.ecg_voltage,
borderColor: "#00FFAA",
backgroundColor: "rgba(0, 255, 170, 0.1)",
borderWidth: 2,
pointRadius: 0,
      tension: 0.4,
    },
  ],
};


const ecgOptions = {
  scales: {
    x: { display: false },
    y: { beginAtZero: false },
  },
  plugins: {
    tooltip: { enabled: true },
    legend: { display: false },
```

```
    },
maintainAspectRatio: false,
    animation: {
      duration: 1000,
      easing: "easeInOutQuad",
    },
  };

  if (data.loading) {
    return <div className="vital-container">Loading...</div>;
  }

  if (data.error) {
    return <div className="vital-container">Error: {data.error}</div>;
  }

  return (
<div  className="vital-container">
<h2 className="vital-title">VitalGuard: Real-Time Monitoring</h2>
<div className="timestamp">Last Updated: {data.timestamp || "Awaiting data..."} (IST)</div>
<div  className="vital-grid">
<div className="vital-card temperature">
<span  className="vital-label">Temperature</span>
<span  className="vital-value">
          {typeofdata.temperature === "number" ?data.temperature.toFixed(1) : "--"} °C
</span>
```

```
</div>
<div className="vital-card spo2">
<span  className="vital-label">SpO2</span>
<span  className="vital-value">
        {typeofdata.spo2 === "number" ?data.spo2.toFixed(1) : "--"} %
</span>
</div>
<div className="vital-card bpm">
<span className="vital-label">Heart Rate</span>
<span  className="vital-value">
        {typeofdata.bpm === "number" ?data.bpm.toFixed(0) : "--"} BPM
</span>
</div>
<div className="vital-card avg-bpm">
<span className="vital-label">Avg BPM</span>
<span  className="vital-value">
        {typeofdata.avg_bpm === "number" ?data.avg_bpm.toFixed(0) : "--"} BPM
</span>
</div>
</div>
<div  className="ecg-container">
<h3 className="ecg-title">ECG Waveform</h3>
<div  className="ecg-chart">
<Line data={ecgData} options={ecgOptions} />
</div>
</div>
```

```
</div>
  );
};

export default VitalSigns;
```

Arduino codes

ECG code:

```
#define ECG_PIN 34  // AD8232 Output to ESP32 GPIO 34

const int samplingRate = 1000;  // 1000 Hz (1 sample per ms)
const int bufferSize = 800;     // Store 500 values for smooth visualization
int ecgBuffer[bufferSize];      // Circular buffer for ECG values
int bufferIndex = 0;            // Index for buffer
unsigned long startTime;        // Track start time

void setup() {
Serial.begin(115200);
analogReadResolution(12);  // 12-bit ADC (0-4095) for better accuracy
startTime = millis();
}

void loop() {
  int rawECG = analogRead(ECG_PIN);  // Read ECG signal
```

```
  // Store in circular buffer
ecgBuffer[bufferIndex] = rawECG;
bufferIndex = (bufferIndex + 1) % bufferSize;  // Loop buffer


  unsigned long elapsedTime = (millis() - startTime) / 1000;  // Time in seconds


  // Print time and ECG value
Serial.print(elapsedTime);
Serial.print(",");
Serial.println(rawECG);


delayMicroseconds(10000);  // 1 ms delay → 1000 Hz sampling rate
}
```

Heartrate SPO2 and temperature code:

```
#include <Adafruit_Sensor.h>
#include <DHT.h>
#include <DHT_U.h>
#include <Wire.h>
#include "MAX30105.h"
#include "heartRate.h"
#include "spo2_algorithm.h"

// DHT11 Temperature Sensor Setup
#define DHTPIN 4
```

```
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);


// MAX30102 Pulse Oximeter & Heart Rate Sensor Setup

MAX30105 particleSensor;


const byte RATE_SIZE = 20;

byte rates[RATE_SIZE];

byte rateSpot = 0;

long lastBeat = 0;


float beatsPerMinute;

int beatAvg;


#define BUFFER_SIZE 100

uint32_t buffer_red[BUFFER_SIZE];

uint32_t buffer_ir[BUFFER_SIZE];

int buffer_counter = 0;

int32_t spo2;

int8_t valid_spo2;

int32_t heart_rate;

int8_t valid_heart_rate;


unsigned long lastMeasurementTime = 0;

unsigned long lastTempReadTime = 0;  // Track last DHT11 read time

const int TEMP_READ_INTERVAL = 8000; // Every 8 seconds
```

```
float lastTemperature = NAN;  // Store last temperature reading


void setup() {
Serial.begin(115200);
Serial.println("Initializing  Sensors...");


dht.begin();


   if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
Serial.println("MAX30105 not found. Check wiring and power.");
       while (1);
   }


Serial.println("Place your index finger on the MAX30102 sensor.");
particleSensor.setup();
particleSensor.setPulseAmplitudeRed(0x0A);
particleSensor.setPulseAmplitudeGreen(0);
}


void loop() {
   // *Continuous MAX30102 Monitoring*
   long irValue = particleSensor.getIR();
   long redValue = particleSensor.getRed();


   if (checkForBeat(irValue) == true) {
```

```
        long delta = millis() - lastBeat;
lastBeat = millis();
beatsPerMinute = 60 / (delta / 1000.0);


        if (beatsPerMinute> 40 &&beatsPerMinute< 200) {
            rates[rateSpot++] = (byte)beatsPerMinute;
rateSpot %= RATE_SIZE;


beatAvg = 0;
            for (byte x = 0 ; x < RATE_SIZE ; x++)
beatAvg += rates[x];
beatAvg /= RATE_SIZE;
        }
    }


buffer_red[buffer_counter] = (uint32_t) redValue;
buffer_ir[buffer_counter] = (uint32_t) irValue;
buffer_counter++;


    if (buffer_counter>= BUFFER_SIZE) {
buffer_counter = 0;


maxim_heart_rate_and_oxygen_saturation(
buffer_ir, BUFFER_SIZE, buffer_red, &spo2, &valid_spo2, &heart_rate, &valid_heart_rate
        );
    }
```

```
    // 🟩 *Non-Blocking DHT11 Temperature Reading (Every 8 Seconds)*
    if (millis() - lastTempReadTime>= TEMP_READ_INTERVAL) {
lastTempReadTime = millis();
        float temp = dht.readTemperature();
        if (!isnan(temp)) {
lastTemperature = temp;
        }
    }

    // 🔹 *Display All Values Every 8 Seconds*
    if (millis() - lastMeasurementTime>= TEMP_READ_INTERVAL) {
lastMeasurementTime = millis();

Serial.print("Temperature:  ");
        if (isnan(lastTemperature)) {
Serial.print("Failed to read DHT11!");
        } else {
Serial.print(lastTemperature);
Serial.print(" °C, ");
        }

Serial.print("BPM=");
Serial.print(beatsPerMinute);
Serial.print(", Avg BPM=");
```

```
Serial.print(beatAvg);


      if (irValue< 50000)
Serial.print(" (No Finger Detected)");


Serial.print(", SpO2=");
      if (valid_spo2)
Serial.print(spo2);
      else
Serial.print("Invalid");


Serial.println();
   }


delay(10);  // Small delay to allow smooth execution
}
```

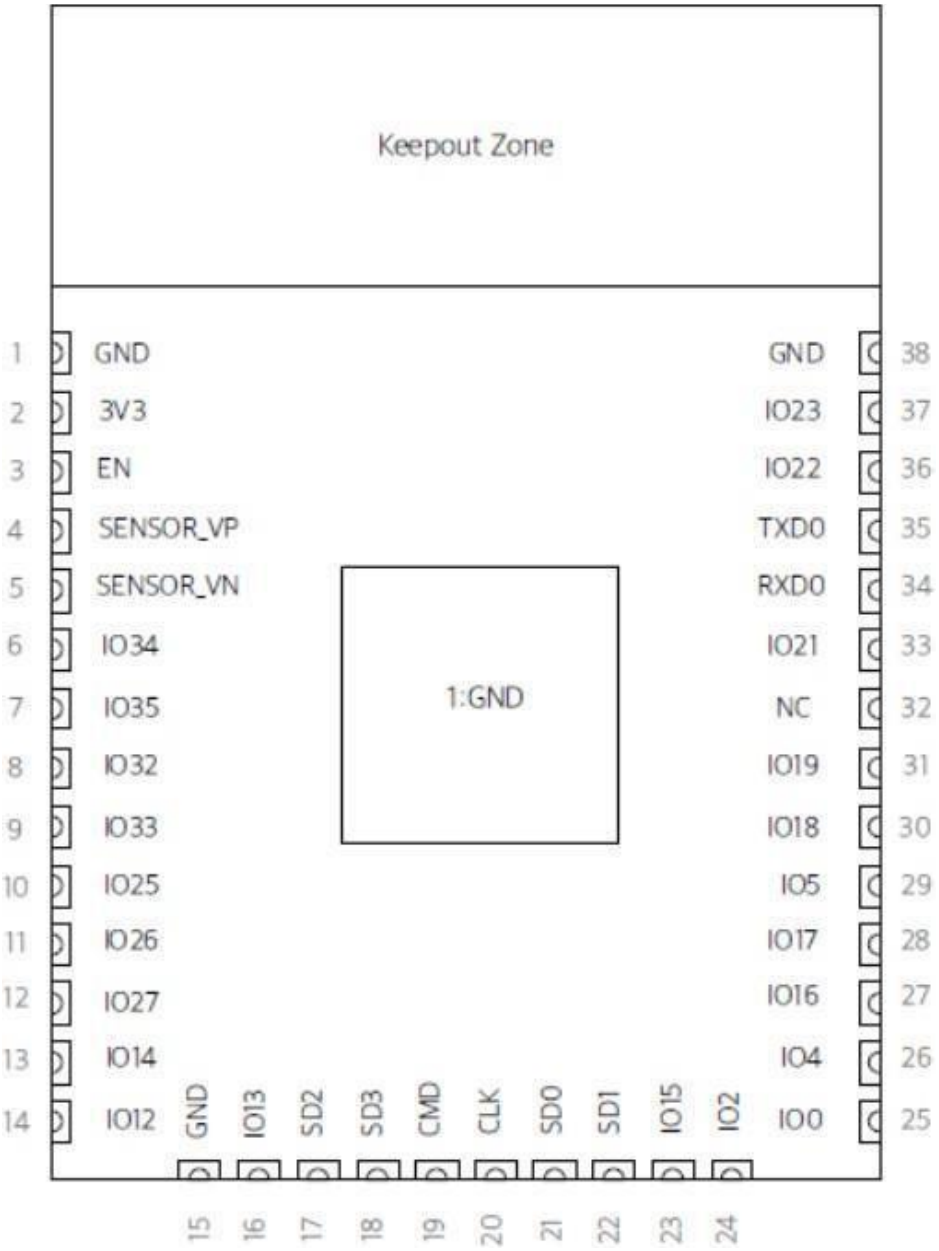
Bill of Components

No	Components	Price
1	ESP32	400
2	MAX30102	300
3	AD8232	790
4	DHT11	120
5	ZERO PCB	40
6	Jumping wires	200

ESP32

Categories	Items	Specifications
Certification	RF certification	FCC/CE/IC/TELEC/KCC/SRRC/NCC
	Wi-Fi certification	Wi-Fi Alliance
	Bluetooth certification	BQB
	Green certification	RoHS/REACH
Wi-Fi	Protocols	802.11 b/g/n (802.11n up to 150 Mbps)
		A-MPDU and A-MSDU aggregation and 0.4 μs guard interval support
	Frequency range	2.4 GHz ~ 2.5 GHz
Bluetooth	Protocols	Bluetooth v4.2 BR/EDR and BLE specification
	Radio	NZIF receiver with -97 dBm sensitivity
		Class-1, class-2 and class-3 transmitter
		AFH
	Audio	CVSD and SBC
Hardware	Module interface	SD card, UART, SPI, SDIO, I2C, LED PWM, Motor PWM, I2S, IR
		GPIO, capacitive touch sensor, ADC, DAC
	On-chip sensor	Hall sensor, temperature sensor
	On-board clock	40 MHz crystal
	Operating voltage/Power supply	2.7 ~ 3.6V
	Operating current	Average: 80 mA
	Minimum current delivered by power supply	500 mA
	Operating temperature range	-40°C ~ +85°C
	Ambient temperature range	Normal temperature
	Package size	18±0.2 mm x 25.5±0.2 mm x 3.1±0.15 mm
Software	Wi-Fi mode	Station/SoftAP/SoftAP+Station/P2P
	Wi-Fi Security	WPA/WPA2/WPA2-Enterprise/WPS
	Encryption	AES/RSA/ECC/SHA
	Firmware upgrade	UART Download / OTA (download and write firmware via network or host)
	Software development	Supports Cloud Server Development / SDK for custom firmware development
	Network protocols	IPv4, IPv6, SSL, TCP/UDP/HTTP/FTP/MQTT
	User configuration	AT instruction set, cloud server, Android/iOS app

Pin Layout

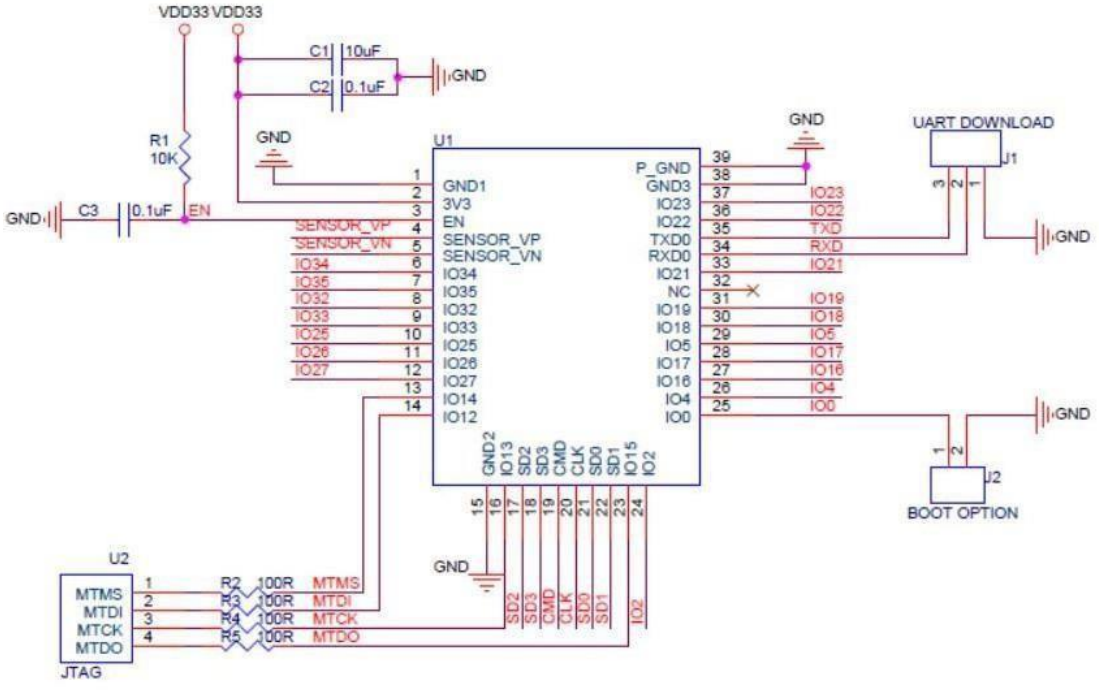


Pin Description

Name	No.	Type	Function
GND	1	P	Ground
3V3	2	P	Power supply.
EN	3	I	Chip-enable signal. Active high.
SENSOR_VP	4	I	GPIO36, SENSOR_VP, ADC_H, ADC1_CH0, RTC_GPIO0
SENSOR_VN	5	I	GPIO39, SENSOR_VN, ADC1_CH3, ADC_H, RTC_GPIO3
IO34	6	I	GPIO34, ADC1_CH6, RTC_GPIO4
IO35	7	I	GPIO35, ADC1_CH7, RTC_GPIO5
IO32	8	I/O	GPIO32, XTAL_32K_P (32.768 kHz crystal oscillator input), ADC1_CH4, TOUCH9, RTC_GPIO9
IO33	9	I/O	GPIO33, XTAL_32K_N (32.768 kHz crystal oscillator output), ADC1_CH5, TOUCH8, RTC_GPIO8
IO25	10	I/O	GPIO25, DAC_1, ADC2_CH8, RTC_GPIO6, EMAC_RXD0
IO26	11	I/O	GPIO26, DAC_2, ADC2_CH9, RTC_GPIO7, EMAC_RXD1
IO27	12	I/O	GPIO27, ADC2_CH7, TOUCH7, RTC_GPIO17, EMAC_RX_DV
IO14	13	I/O	GPIO14, ADC2_CH6, TOUCH6, RTC_GPIO16, MTMS, HSPICLK, HS2_CLK, SD_CLK, EMAC_TXD2
IO12	14	I/O	GPIO12, ADC2_CH5, TOUCH5, RTC_GPIO15, MTDI, HSPIQ, HS2_DATA2, SD_DATA2, EMAC_TXD3
GND	15	P	Ground
IO13	16	I/O	GPIO13, ADC2_CH4, TOUCH4, RTC_GPIO14, MTCK, HSPID, HS2_DATA3, SD_DATA3, EMAC_RX_ER
SHD/SD2*	17	I/O	GPIO9, SD_DATA2, SPIHD, HS1_DATA2, U1RXD
SWP/SD3*	18	I/O	GPIO10, SD_DATA3, SPIWP, HS1_DATA3, U1TXD
SCS/CMD*	19	I/O	GPIO11, SD_CMD, SPICS0, HS1_CMD, U1RTS
SCK/CLK*	20	I/O	GPIO6, SD_CLK, SPICLK, HS1_CLK, U1CTS
SDO/SD0*	21	I/O	GPIO7, SD_DATA0, SPIQ, HS1_DATA0, U2RTS
SDI/SD1*	22	I/O	GPIO8, SD_DATA1, SPID, HS1_DATA1, U2CTS
IO15	23	I/O	GPIO15, ADC2_CH3, TOUCH3, MTDO, HSPICS0, RTC_GPIO13, HS2_CMD, SD_CMD, EMAC_RXD3
IO2	24	I/O	GPIO2, ADC2_CH2, TOUCH2, RTC_GPIO12, HSPIWP, HS2_DATA0, SD_DATA0

Name	No.	Type	Function
IO0	25	I/O	GPIO0, ADC2_CH1, TOUCH1, RTC_GPIO11, CLK_OUT1, EMAC_TX_CLK
IO4	26	I/O	GPIO4, ADC2_CH0, TOUCH0, RTC_GPIO10, HSPIHD, HS2_DATA1, SD_DATA1, EMAC_TX_ER
IO16	27	I/O	GPIO16, HS1_DATA4, U2RXD, EMAC_CLK_OUT
IO17	28	I/O	GPIO17, HS1_DATA5, U2TXD, EMAC_CLK_OUT_180
IO5	29	I/O	GPIO5, VSPICS0, HS1_DATA6, EMAC_RX_CLK
IO18	30	I/O	GPIO18, VSPICLK, HS1_DATA7
IO19	31	I/O	GPIO19, VSPIQ, U0CTS, EMAC_TXD0
NC	32	-	-
IO21	33	I/O	GPIO21, VSPIHD, EMAC_TX_EN
RXD0	34	I/O	GPIO3, U0RXD, CLK_OUT2
TXD0	35	I/O	GPIO1, U0TXD, CLK_OUT3, EMAC_RXD2
IO22	36	I/O	GPIO22, VSPIWP, U0RTS, EMAC_TXD1
IO23	37	I/O	GPIO23, VSPID, HS1_STROBE
GND	38	P	Ground

Peripheral Schematic



MTDI should be kept at a low electric level when powering up the module.

DHT11

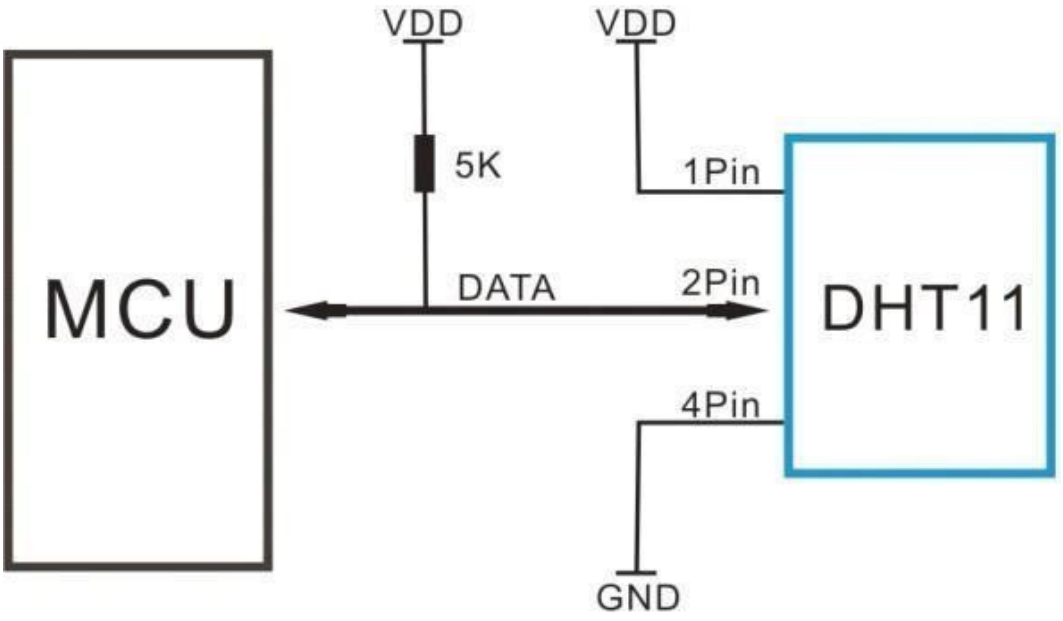
Overview

Item	Measurement Range	Humidity Accuracy	Temperature Accuracy	Resolution	Package
DHT11	20-90%RH 0-50 ℃	±5%RH	±2℃	1	4 Pin Single Row

Detailed specification

Parameters	**Conditions**	**Minimum**	**Typical**	**Maximum**
Humidity				
Resolution		1%RH	1%RH	1%RH
			8 Bit	
Repeatability			±1%RH	
Accuracy	25℃		±4%RH	
	0-50℃			±5%RH
Interchangeability	Fully Interchangeable			
Measurement Range	0℃	30%RH		90%RH
	25℃	20%RH		90%RH
	50℃	20%RH		80%RH
Response Time (Seconds)	1/e(63%)25℃, 1m/s Air	6 S	10 S	15 S
Hysteresis			±1%RH	
Long-Term Stability	Typical		±1%RH/year	
Temperature				
Resolution		1℃	1℃	1℃
		8 Bit	8 Bit	8 Bit
Repeatability			±1℃	
Accuracy		±1℃		±2℃
Measurement Range		0℃		50℃
Response Time (Seconds)	1/e(63%)	6 S		30 S

Typical application



Electrical Characteristics

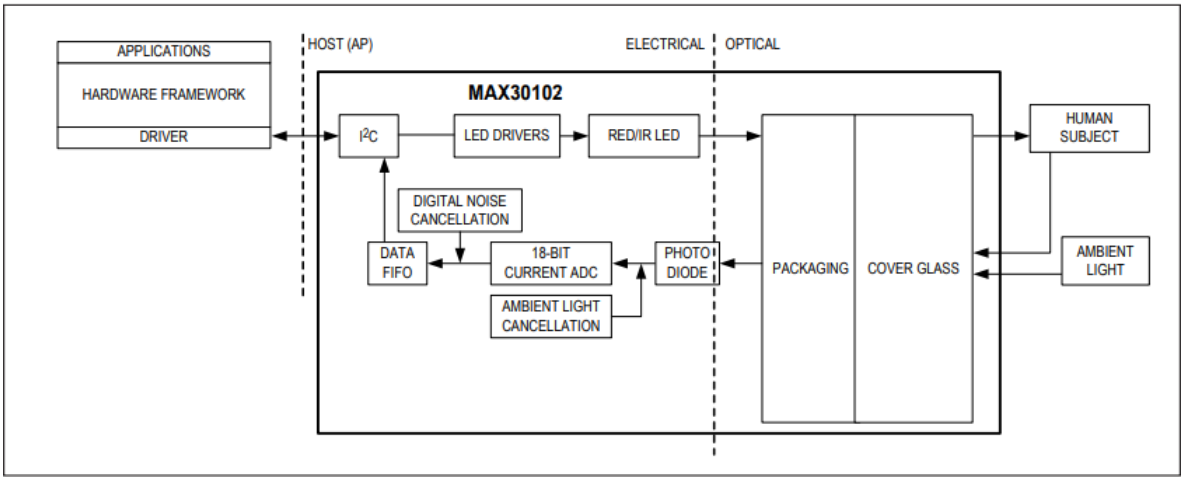
VDD=5V, T = 25℃ (unless otherwise stated)

	Conditions	Minimum	Typical	Maximum
Power Supply	DC	3V	5V	5.5V
Current Supply	Measuring	0.5mA		2.5mA
	Average	0.2mA		1mA
	Standby	100uA		150uA
Sampling period	Second	1		

AD8232

Pin Name	Description
GND	Power Supply Ground
3.3v	Power Supply 3.3v
Output (ADC)	Operational Amplifier Output. The fully conditioned heart rate signal is present at this output. OUT can be connected to the input of an ADC.
LO-	Leads Off Comparator Output. In dc leads off detection mode, LO– is high when the electrode to –IN is disconnected, and it is low when connected
LO+	Leads Off Comparator Output. In dc leads off detection mode, LOD+ is high when the +IN electrode is disconnected, and it is low when connected
SDN	Shutdown Control Input. Drive SDN low to enter the low power shutdown mode.
RA (Right Arm)	**RED Biomedical electrode pad RA(input).** Instrumentation Amplifier Negative Input. –IN is typically connected to the right arm (RA) electrode
RA (Right Arm)	**RED Biomedical electrode pad RA(input).** Instrumentation Amplifier Negative Input. –IN is typically connected to the right arm (RA) electrode
LA (Left Arm)	**YELLOW Biomedical electrode pad LA(input).** Instrumentation Amplifier Positive Input. +IN is typically connected to the left arm (LA) electrode
RL(Right Leg)	**GREEN Biomedical electrode pad RL(input).** Right Leg Drive Output. Connect the driven electrode (typically, right leg) to the RLD pin.
3.5mm ECG Biomedical Electrode Connector Jack	Combine Biomedical Electrode pad Connector of RA, LA, RL

System Design



Electrical Characteristics

(V_{DD} = 1.8V, V_{LED+} = 5.0V, T_A = -40°C to +85°C, unless otherwise noted. Typical values are at T_A = +25°C) (Note 1)

PARAMETER	SYMBOL	CONDITIONS	MIN	TYP	MAX	UNITS
POWER SUPPLY						
Power-Supply Voltage	V_{DD}	Guaranteed by RED and IR count tolerance	1.7	1.8	2.0	V
LED Supply Voltage V_{LED+} to PGND	V_{LED+}	Guaranteed by PSRR of LED driver	3.1	3.3	5.0	V
Supply Current	I_{DD}	SpO_2 and HR mode, PW = 215µs, 50sps		600	1200	µA
		IR only mode, PW = 215µS, 50sps		600	1200	
Supply Current in Shutdown	I_{SHDN}	T_A = +25°C, MODE = 0x80		0.7	10	µA

Electrical Characteristics(continued)

PULSE OXIMETRY/HEART-RATE SENSOR CHARACTERISTICS							
ADC Resolution				18			bits
Red ADC Count (Note 2)	REDC	LED1_PA = 0x0C, LED_PW = 0x01, SPO2_SR = 0x05, ADC_RGE = 0x00		65536			Counts
IR ADC Count (Note 2)	IRC	LED2_PA = 0x0C, LED_PW = 0x01, SPO2_SR = 0x05 ADC_RGE = 0x00		65536			Counts
Dark Current Count	LED_DCC	LED1_PA = LED2_PA = 0x00, LED_PW = 0x03, SPO2_SR = 0x01 ADC_RGE = 0x02			30	128	Counts
					0.01	0.05	% of FS
DC Ambient Light Rejection	ALR	ADC counts with finger on sensor under direct sunlight (100K lux), ADC_RGE = 0x3, LED_PW = 0x03, SPO2_SR = 0x01	Red LED		2		Counts
			IR LED		2		Counts
ADC Count—PSRR (V_{DD})	$PSRRV_{DD}$	1.7V < V_{DD} < 2.0V, LED_PW = 0x01, SPO2_SR = 0x05			0.25	1	% of FS
		Frequency = DC to 100kHz, 100mV_{P-P}			10		LSB
ADC Count—PSRR (LED Driver Outputs)	$PSRR_{LED}$	3.1V < V_{LED+}, < 5.0V, LED1_PA = LED2_PA = 0x0C, LED_PW = 0x01, SPO2_SR = 0x05			0.05	1	% of FS
		Frequency = DC to 100kHz, 100mV_{P-P}			10		LSB
ADC Clock Frequency	CLK			10.32	10.48	10.64	MHz
ADC Integration Time	INT	LED_PW = 0x00			69		µs
		LED_PW = 0x01			118		
		LED_PW = 0x02			215		
		LED_PW = 0x03			411		
Slot Timing (Timing Between Sequential Channel Samples; e.g., Red Pulse Rising Edge To IR Pulse Rising Edge)	INT	LED_PW = 0x00			427.1		µs
		LED_PW = 0x01			524.7		
		LED_PW = 0x02			720.0		
		LED_PW = 0x03			1106.6		

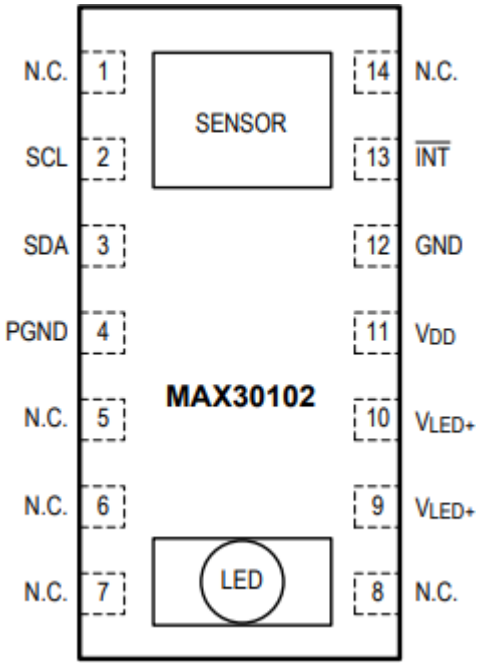
Electrical Characteristics(continued)

PARAMETER	SYMBOL	CONDITIONS	MIN	TYP	MAX	UNITS
IR LED CHARACTERISTICS (Note 3)						
LED Peak Wavelength	λ_P	I_{LED} = 20mA, T_A = +25°C	870	880	900	nm
Full Width at Half Max	$\Delta\lambda$	I_{LED} = 20mA, T_A = +25°C		30		nm
Forward Voltage	V_F	I_{LED} = 20mA, T_A = +25°C		1.4		V
Radiant Power	P_O	I_{LED} = 20mA, T_A = +25°C		6.5		mW
RED LED CHARACTERISTICS (Note 3)						
LED Peak Wavelength	λ_P	I_{LED} = 20mA, T_A = +25°C	650	660	670	nm
Full Width at Half Max	$\Delta\lambda$	I_{LED} = 20mA, T_A = +25°C		20		nm
Forward Voltage	V_F	I_{LED} = 20mA, T_A = +25°C		2.1		V
Radiant Power	P_O	I_{LED} = 20mA, T_A = +25°C		9.8		mW
PHOTODETECTOR CHARACTERISTICS (Note 3)						
Spectral Range of Sensitivity	λ (QE > 50%)	QE: Quantum Efficiency	600		900	nm
Radiant Sensitive Area	A			1.36		mm^2
Dimensions of Radiant Sensitive Area	L x W			1.38 x 0.98		mm x mm
INTERNAL DIE TEMPERATURE SENSOR						
Temperature ADC Acquisition Time	T_T	T_A = +25°C		29		ms
Temperature Sensor Accuracy	T_A	T_A = +25°C		±1		°C
Temperature Sensor Minimum Range	T_{MIN}			-40		°C
Temperature Sensor Maximum Range	T_{MAX}			85		°C
DIGITAL INPUT CHARACTERISTICS: SCL, SDA						
Input High Voltage	V_{IH}	V_{DD} = 2V	0.7 x V_{DD}			V
Input Low Voltage	V_{IL}	V_{DD} = 2V			0.3 x V_{DD}	V
Hysteresis Voltage	V_H			0.2		V
Input Leakage Current	I_{IN}	V_{IN} = GND or V_{DD} (STATIC)		±0.05	±1	µA
DIGITAL OUTPUT CHARACTERISTICS: SDA, $\overline{INT}$						
Ouput Low Voltage	V_{OL}	I_{SINK} = 6mA			0.2	V

Electrical Characteristics(continued)

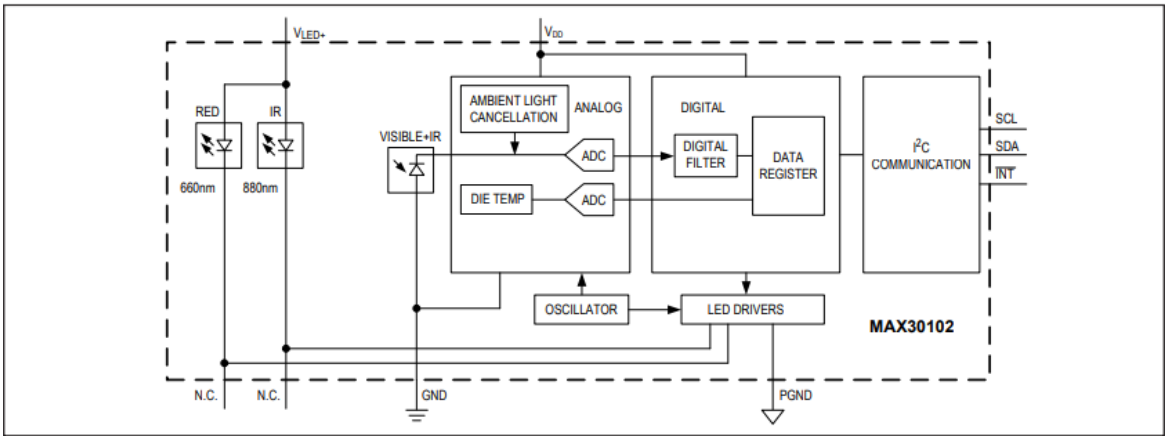
PARAMETER	SYMBOL	CONDITIONS	MIN	TYP	MAX	UNITS
I²C TIMING CHARACTERISTICS (SDA, SDA, $\overline{INT}$) (Note 3)						
I²C Write Address				AE		Hex
I²C Read Address				AF		Hex
Serial Clock Frequency	f_{SCL}		0		400	kHz
Bus Free Time Between STOP and START Conditions	t_{BUF}		1.3			µs
Hold Time (Repeated) START Condition	$t_{HD;STA}$		0.6			µs
SCL Pulse-Width Low	t_{LOW}		1.3			µs
SCL Pulse-Width High	t_{HIGH}		0.6			µs
Setup Time for a Repeated START Condition	$t_{SU;STA}$		0.6			µs
Data Hold Time	$t_{HD;DAT}$		0		900	ns
Data Setup Time	$t_{SU;DAT}$		100			ns
Setup Time for STOP Condition	$t_{SU;STO}$		0.6			µs
Pulse Width of Suppressed Spike	t_{SP}		0		50	ns
Bus Capacitance	C_B				400	pF
SDA and SCL Receiving Rise Time	t_R		$20 + 0.1C_B$		300	ns
SDA and SCL Receiving Fall Time	t_{RF}		$20 + 0.1C_B$		300	ns
SDA Transmitting Fall Time	t_{TF}				300	ns

Pin Configuration



PIN	NAME	FUNCTION
1, 5, 6, 7, 8, 14	N.C.	No Connection. Connect to PCB pad for mechanical stability.
2	SCL	I²C Clock Input
3	SDA	I²C Data, Bidirectional (Open-Drain)
4	PGND	Power Ground of the LED Driver Blocks
9	V_{LED+}	LED Power Supply (anode connection). Use a bypass capacitor to PGND for best performance.
10	V_{LED+}	
11	V_{DD}	Analog Power Supply Input. Use a bypass capacitor to GND for best performance.
12	GND	Analog Ground
13	$\overline{INT}$	Active-Low Interrupt (Open-Drain). Connect to an external voltage with a pullup resistor.

Functional Diagram



10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **79 Not Cited or Quoted 10%**
  Matches with neither in-text citation nor quotation marks
- **0 Missing Quotations 0%**
  Matches that are still very similar to source material
- **0 Missing Citation 0%**
  Matches that have quotation marks, but no in-text citation
- **0 Cited and Quoted 0%**
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
7% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 79 Not Cited or Quoted 10%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
7% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
www.coursehero.com		3%
2	Publication	
XiaoMin Zhu, Donghua Chen, Ying Chen, Huisheng Chen. "A resource integration ...		3%
3	Publication	
"IEEE-ICDCS conference proceeding", 2012 International Conference on Devices Ci...		1%
4	Publication	
Phani Kumar Polasi, Aishwarya S, Kruthika P, Mohammed Kaif Momin. "An IoT-ba...		<1%
5	Publication	
Muhammad Raza Naqvi. "Low power network on chip architectures: A survey", C...		<1%
6	Internet	
www.jetir.org		<1%
7	Internet	
fastercapital.com		<1%
8	Internet	
science.lpnu.ua		<1%
9	Publication	
Davis, Anna K.. "Female Criminal Agency in Early Fourteenth-Century Norfolk", Te...		<1%
10	Internet	
www.farnell.com		<1%

11	Internet	
www.ijert.org		<1%
12	Publication	
S Sooraj, Ancy S Anselam, Sakuntala S Pillai. "Performance analysis of CELP codec ...		<1%
13	Internet	
amsec.wwu.edu		<1%
14	Internet	
www.scribd.com		<1%
15	Internet	
datasheetspdf.com		<1%
16	Internet	
www.adafruit.com		<1%
17	Publication	
"Artificial Intelligence Technologies and Applications", IOS Press, 2024		<1%
18	Publication	
Hanoi Pedagogical University 2		<1%
19	Publication	
Nilanjan Dey, Girish Mishra, Bijurika Nandi, Moumita Pal, Achintya Das, Sheli Sinh...		<1%
20	Internet	
docshare.tips		<1%
21	Publication	
Dhananjay Gadre. "Programming the Parallel Port - Interfacing the PC for Data A...		<1%
22	Publication	
P. Baraneedharan, S. Kalaivani, S. Vaishnavi, K. Somasundaram. "Revolutionizing ...		<1%